UNIVERSIDAD DE LA GUAJIRA

Facultad de Ingenierías - Ingeniería de Sistemas

Parcial 3 - Ingeniería de Software I

Caso EnvíaYa: pruebas, mantenimiento y evolución

Estudiante: Álvaro Javier Montero Fonseca

Código estudiantil: 0182310065

Grupo: B1 - VI semestre

Docente: Ing. Robert Damián Quintero Laverde

Fecha: 11 de noviembre de 2026

Repositorio: github.com/alvarojaviermonterofonseca-ux/parcial-3-de-inegenieria-de-software

Mis parámetros

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
3	1	0	0	6	5	45 kg	3 kg	10 kg	20 %	50 %	10 %	$2.000	2 (hora pico)

D = 0 (par)	E = 6 (par)	B = 1 (impar)	F = 5 (impar)
El tramo 1 incluye el límite: peso ≤ 3	El recargo de fin de semana va antes de los descuentos	Se aplican ambos descuentos: cliente y luego cupón	Un cupón que no existe es error

Parte 1 - Requisitos y criterios de aceptación

1.1 Mis parámetros

Código estudiantil **0182310065**. Últimos seis dígitos: **3 1 0 0 6 5**.

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
3	1	0	0	6	5	45 kg	3 kg	10 kg	20 %	50 %	10 %	$2.000	2

Cómo sale cada valor:

Parámetro	Fórmula	Cálculo	Valor
PMAX	30 + 5·A	30 + 5·3	**45 kg**
L1	2 + (B mod 3)	2 + (1 mod 3) = 2 + 1	**3 kg**
L2	10 + C	10 + 0	**10 kg**
RR	10 + 5·(E mod 4)	10 + 5·(6 mod 4) = 10 + 5·2	**20 %**
RN	40 + 5·(F mod 3)	40 + 5·(5 mod 3) = 40 + 5·2	**50 %**
V	10 + 5·(A mod 3)	10 + 5·(3 mod 3) = 10 + 0	**10 %**
S	2000 + 500·(C mod 4)	2000 + 500·0	**$2.000**
CAMBIO	(A + F) mod 3	(3 + 5) mod 3 = 8 mod 3	**2 - hora pico**

Reglas que dependen de la paridad:

Dígito	Valor	Paridad	Regla que me corresponde
D - tramo 1	0	par	El tramo 1 **incluye** el límite: peso ≤ 3 kg
E - recargo de fin de semana	6	par	Se suma **antes** de los descuentos
B - cliente y cupón	1	impar	Se aplican **ambos**: primero el del cliente y luego el cupón
F - cupón que no existe	5	impar	**Es error**

1.2 Especificación completa (R1 a R8 con mis valores)

La función recibe: `peso` en kg (puede tener decimales); `zona` (urbana, rural o nacional); `cliente` (normal, frecuente o vip); `dia` de 1 a 7, donde 6 es sábado y 7 es domingo; y `cupon`, que es opcional.

ID	Regla
R1	El peso debe cumplir **0 < peso ≤ 45**. Si no, se lanza el error `"peso inválido"`.
R2	Si la zona no es `urbana`, `rural` o `nacional`, o el día no está entre 1 y 7, se lanza un error.
R3	Tarifa base. Tramo 1 (**peso ≤ 3 kg**): $6.000. Tramo 2 (3 < peso ≤ 10 kg): $9.000. Más de 10 kg: $9.000 + $800 por cada kg sobre 10, de forma proporcional.
R4	Recargo por zona. Rural: **+20 %**. Nacional: **+50 %**. Urbana: sin recargo.
R5	Fin de semana (día 6 o 7): **+$2.000**, que se suma **antes** de aplicar los descuentos.
R6	Descuento de cliente. Frecuente: 5 %. VIP: **10 %**. Cupón `ENVIO10`: 10 %. Si hay los dos, **se aplican ambos**: primero el del cliente y luego el cupón, cada uno sobre el valor que deja el anterior.
R7	Un cupón que no existe **es error**.
R8	El costo nunca es menor que $5.000. El resultado se redondea al múltiplo de $100 más cercano; los $50 redondean hacia arriba.

Orden del cálculo que resulta de mis reglas:

1. Validar peso, zona, día y cupón (R1, R2, R7).
2. Tarifa base según el tramo (R3).
3. Recargo por zona (R4).
4. Recargo de fin de semana (R5).

5. Descuento del cliente y después el del cupón (R6).
6. Costo mínimo y redondeo a $100 (R8).

Supuesto: la especificación no dice qué pasa si el tipo de cliente no es uno de los tres válidos. No lo traté como error porque ninguna regla lo pide; un cliente desconocido se cobra sin descuento, igual que en el código heredado. Queda como pregunta abierta para el dueño del producto.

1.3 Historias de usuario

HU-01 - Validar los datos del envío (R1, R2)

> **Como** operador de mostrador de EnvíaYa, **quiero** que el sistema rechace los envíos con peso, zona o día que no son válidos, **para** no cobrar ni despachar envíos que la empresa no puede atender.

Criterios de aceptación:

- **CA-01.1 Dado** un envío de 45 kg a zona urbana un lunes, **cuando** calculo el costo, **entonces** el sistema lo acepta y devuelve $37.000.
- **CA-01.2 Dado** un envío de 45,01 kg, **cuando** calculo el costo, **entonces** el sistema lanza el error "peso inválido".
- **CA-01.3 Dado** un envío de 0 kg o de peso negativo, **cuando** calculo el costo, **entonces** el sistema lanza el error "peso inválido".
- **CA-01.4 Dado** un envío a la zona "internacional", **cuando** calculo el costo, **entonces** el sistema lanza un error y no devuelve ningún valor.
- **CA-01.5 Dado** un envío con día 0 o día 8, **cuando** calculo el costo, **entonces** el sistema lanza un error.

HU-02 - Cobrar según el peso y la zona (R3, R4, R8)

> **Como** cliente de EnvíaYa, **quiero** que el costo dependa del peso del paquete y de la zona de destino, **para** pagar un valor justo y saber de antemano cuánto cuesta mi envío.

Criterios de aceptación:

- **CA-02.1 Dado** un paquete de 3 kg a zona urbana un lunes, **cuando** calculo el costo, **entonces** se cobra el tramo 1: $6.000.
- **CA-02.2 Dado** un paquete de 3,01 kg a zona urbana un lunes, **cuando** calculo el costo, **entonces** se cobra el tramo 2: $9.000.
- **CA-02.3 Dado** un paquete de 20 kg a zona urbana un lunes, **cuando** calculo el costo, **entonces** se cobra $9.000 + 10 × $800 = $17.000.
- **CA-02.4 Dado** un paquete de 5 kg, **cuando** va a zona rural, **entonces** cuesta $10.800 (+20 %); y **cuando** va a zona nacional, **entonces** cuesta $13.500 (+50 %).
- **CA-02.5 Dado** un envío cuyo cálculo da $4.860, **cuando** se entrega el resultado, **entonces** se cobra el mínimo de $5.000.
- **CA-02.6 Dado** un envío cuyo cálculo da $8.550, **cuando** se entrega el resultado, **entonces** se redondea hacia arriba a $8.600.

HU-03 - Aplicar el recargo de fin de semana y los descuentos (R5, R6, R7)

> **Como** gerente comercial de EnvíaYa, **quiero** que se cobre el recargo de fin de semana y que se apliquen los descuentos de cliente y de cupón, **para** cubrir el costo de operar sábados y domingos y premiar a los clientes fieles sin regalar descuentos que no existen.

Criterios de aceptación:

- **CA-03.1 Dado** un envío de 5 kg urbano de un cliente normal, **cuando** se hace un sábado o un domingo, **entonces** cuesta $11.000 ($9.000 + $2.000).
- **CA-03.2 Dado** un envío de 5 kg urbano de un cliente VIP un domingo, **cuando** calculo el costo, **entonces** el recargo se suma antes del descuento: ($9.000 + $2.000) × 0,90 = $9.900.
- **CA-03.3 Dado** un cliente VIP con el cupón ENVIO10 y un envío de 5 kg urbano un lunes, **cuando** calculo el costo, **entonces** se aplican los dos descuentos: $9.000 × 0,90 × 0,90 = $7.290, que se redondea a $7.300.
- **CA-03.4 Dado** un cliente frecuente sin cupón y un envío de 5 kg urbano un lunes, **cuando** calculo el costo, **entonces** se descuenta el 5 %: $8.550, que se redondea a $8.600.
- **CA-03.5 Dado** un envío con el cupón "NOEXISTE", **cuando** calculo el costo, **entonces** el sistema lanza un error.

1.4 Matriz de trazabilidad

Los casos `CN-xx` son de caja negra (Parte 3) y los `CB-xx` de caja blanca (Parte 4). Los `HP-xx` se agregan en la Parte 8.

Requisito	Historia	Criterios	Casos de prueba
R1 peso válido	HU-01	CA-01.1, CA-01.2, CA-01.3	CN-01, CN-02, CN-03, CN-12, CN-13, CN-14, CN-42, CB-01, CB-02
R2 zona y día válidos	HU-01	CA-01.4, CA-01.5	CN-17, CN-18, CN-19, CN-23, CB-08
R3 tarifa base	HU-02	CA-02.1, CA-02.2, CA-02.3	CN-03 a CN-12, CB-03, CB-04, CB-05
R4 recargo por zona	HU-02	CA-02.4	CN-15, CN-16, CN-38, CN-39, CN-40, CN-41, CB-06, CB-07
R5 fin de semana	HU-03	CA-03.1, CA-03.2	CN-20, CN-21, CN-22, CN-29 a CN-33, CN-38, CN-39, CN-41, CB-09
R6 descuentos	HU-03	CA-03.3, CA-03.4	CN-24 a CN-33, CN-37, CN-38, CN-40, CB-10, CB-11, CB-12
R7 cupón inexistente	HU-03	CA-03.5	CN-34, CN-35, CB-13
R8 mínimo y redondeo	HU-02	CA-02.5, CA-02.6	CN-09, CN-11, CN-25, CN-26, CN-28, CN-30, CN-31, CN-36, CN-37, CB-14

Parte 2 - Plan de pruebas

2.1 Alcance

Objeto de prueba: la función `calc` del módulo `envios.py` de EnvíaYa, versión heredada 1.0.0 y las versiones que salgan del mantenimiento.

Dentro del alcance

- Las ocho reglas de negocio R1 a R8 con los valores del código 0182310065.
- Validación de entradas: peso, zona, día y cupón.
- Cálculo completo: tarifa base, recargo de zona, recargo de fin de semana, descuentos, costo mínimo y redondeo.
- Pruebas de regresión después de cada corrección y de la refactorización.
- El cambio de la Parte 8 (hora pico).

Fuera del alcance

- Interfaz gráfica, base de datos, facturación y cualquier otro módulo del sistema: solo se entregó la función de cálculo.
- Pruebas de rendimiento, carga y seguridad: la función es un cálculo en memoria sin entrada ni salida.
- Tipos de dato no numéricos en el peso o el día (por ejemplo, texto): la especificación no los define.

2.2 Niveles de prueba aplicados al caso

Nivel	Qué se prueba aquí	Cómo se hace	Quién
Unidad	La función `calc` aislada y, después de refactorizar, cada función extraída: validación, tarifa base, estrategia de zona, descuentos y redondeo.	pytest con casos de caja negra (`tests/test_caja_negra.py`) y de caja blanca (`tests/test_caja_blanca.py`); cobertura con pytest-cov.	Desarrollador / tester
Integración	Que las piezas trabajen juntas en el orden correcto: tarifa → zona → fin de semana → descuentos → mínimo → redondeo. Tras la refactorización, que `calc` combine bien las estrategias de zona con las funciones de descuento.	Los casos combinados CN-38 a CN-41 recorren todas las reglas en una sola llamada. La integración es ascendente: primero las funciones pequeñas, luego `calc`.	Tester
Sistema	El módulo completo, visto como lo usaría el sistema de mostrador: se llama a `calc` con datos de envíos reales y se compara con la tarifa oficial.	Ejecución de toda la suite desde la línea de comandos (`python -m pytest`) en un ambiente limpio, sobre la versión etiquetada.	Tester
Aceptación	Que el cálculo cumple lo que el negocio pidió.	Los criterios Dado / Cuando / Entonces de las historias HU-01 a HU-03 se revisan con el dueño del producto; cada criterio tiene al menos un caso automatizado en la matriz de trazabilidad.	Dueño del producto (gerente comercial) con apoyo del tester

2.3 Estrategia

1. **Pruebas basadas en la especificación (caja negra):** particiones de equivalencia, valores límite y tabla de decisión. Los resultados esperados se calculan a mano con la especificación, nunca ejecutando el código.
2. **Pruebas basadas en la estructura (caja blanca):** grafo de flujo, complejidad ciclomática y caminos independientes para asegurar cobertura de sentencias y de ramas.
3. **Automatización:** todos los casos se escriben en pytest para poder repetirlos sin costo.
4. **Primero contra el código heredado:** la suite se ejecuta contra la versión 1.0.0 sin tocarla; cada fallo se analiza y se registra como defecto.

5. **Regresión:** después de cada corrección (`fix/DEF-xx`) y de la refactorización se ejecuta de nuevo la suite completa. Ninguna rama se integra a `main` si rompe una prueba que antes pasaba.
6. **Pruebas primero en los cambios:** para la solicitud de cambio se escribe la prueba, se ve fallar y luego se implementa.

2.4 Ambiente

Elemento	Valor
Sistema operativo	Windows 11
Lenguaje	Python 3.14
Marco de pruebas	pytest 9
Cobertura	pytest-cov (coverage.py), con cobertura de ramas
Complejidad	radon
Control de versiones	Git y GitHub
Datos de prueba	Valores escritos en los propios casos; no hay base de datos

Comando de ejecución:

```
python -m pytest --cov=envios --cov-branch --cov-report=term-missing
```

2.5 Roles

El trabajo es individual, así que una sola persona cumple todos los roles:

Rol	Responsabilidad
Analista de pruebas	Escribe la especificación, las historias y diseña los casos.
Tester / automatizador	Programa los casos en pytest, los ejecuta y reporta los defectos.
Desarrollador de mantenimiento	Corrige los defectos y refactoriza.
Gestor de configuración	Maneja ramas, etiquetas, versiones y el CHANGELOG.
Dueño del producto (docente)	Aclara dudas de la especificación y acepta la entrega.

2.6 Riesgos y mitigación

#	Riesgo	Prob.	Impacto	Mitigación
1	Calcular mal un resultado esperado a mano y reportar un defecto que no existe.	Media	Alto	Mostrar el cálculo paso a paso; revisar cada fallo contra la especificación antes de registrarlo.
2	Usar parámetros que no son los de mi código estudiantil.	Baja	Muy alto (nota 0)	Tabla de parámetros con el cálculo de cada fórmula al inicio; CONFIG revisado contra esa tabla.
3	Que una corrección dañe algo que funcionaba (regresión).	Media	Alto	Suite completa después de cada rama; integrar solo en verde.
4	Errores de punto flotante en el redondeo (por ejemplo 8.549,999 en lugar de 8.550).	Media	Medio	Casos que caen justo en $50; en la refactorización el dinero se maneja con `Decimal`.
5	Ambigüedades de la especificación (cliente desconocido, cupón vacío).	Media	Medio	Dejarlas escritas como supuestos y no inventar reglas.
6	Que la refactorización cambie el comportamiento sin querer.	Media	Alto	Refactorizar en pasos pequeños con las mismas pruebas como red de seguridad.
7	Poco tiempo: la entrega es el mismo día.	Alta	Alto	Priorizar por rúbrica; commits pequeños y frecuentes para no perder trabajo.

2.7 Criterios de entrada

- Especificación R1 a R8 con mis valores escrita y revisada.

- Código heredado en el repositorio con la etiqueta `v1.0.0` y `CONFIG` lleno.
- Ambiente instalado: Python, pytest y pytest-cov funcionando.
- Casos de prueba diseñados con su resultado esperado calculado a mano.

2.8 Criterios de salida

- El 100 % de los casos diseñados está automatizado y ejecutado.
- El 100 % de los casos pasa sobre la versión corregida.
- Cobertura de sentencias y de ramas del 100 % en `envios.py`.
- Ningún defecto de severidad alta o media abierto.
- Cada defecto encontrado tiene su reporte, su rama `fix/DEF-xx` y una prueba que lo reproduce.
- Versión publicada con etiqueta, CHANGELOG y notas de versión.

2.9 Entregables

Plan de pruebas (este documento), diseño de casos (Partes 3 y 4), código de las pruebas en `tests/`, reporte de defectos y métricas (Parte 5), evidencias de ejecución en `docs/evidencias/` y el informe final en PDF.

Parte 3 - Diseño de casos de caja negra

Todos los resultados esperados salen de la especificación (Parte 1) con mis valores: PMAX = 45, L1 = 3 (incluido), L2 = 10, rural +20 %, nacional +50 %, VIP 10 %, fin de semana +$2.000 antes de los descuentos, ambos descuentos y cupón inexistente = error.

3.1 Particiones de equivalencia

Peso (kg)

Clase	Tipo	Rango	Representante	Resultado
P1	Inválida	peso ≤ 0	−1	Error "peso inválido"
P2	Válida	0 < peso ≤ 3 (tramo 1)	2	Base $6.000
P3	Válida	3 < peso ≤ 10 (tramo 2)	5	Base $9.000
P4	Válida	10 < peso ≤ 45 (tramo 3)	20	Base $9.000 + $800 × (peso − 10)
P5	Inválida	peso > 45	50	Error "peso inválido"

Zona

Clase	Tipo	Valor	Resultado
Z1	Válida	`urbana`	Sin recargo
Z2	Válida	`rural`	+20 %
Z3	Válida	`nacional`	+50 %
Z4	Inválida	cualquier otro texto (`internacional`)	Error

Cliente

Clase	Tipo	Valor	Resultado
C1	Válida	`normal`	Sin descuento
C2	Válida	`frecuente`	−5 %
C3	Válida	`vip`	−10 %
C4	No definida	otro texto	La especificación no lo define (ver supuesto de la Parte 1); no se diseñó caso con resultado esperado.

Día

Clase	Tipo	Rango	Representante	Resultado
D1	Inválida	día < 1	0	Error
D2	Válida	1 a 5 (día hábil)	1 y 5	Sin recargo
D3	Válida	6 y 7 (fin de semana)	6 y 7	+$2.000
D4	Inválida	día > 7	8	Error

Cupón

Clase	Tipo	Valor	Resultado
K1	Válida	sin cupón	Sin descuento de cupón
K2	Válida	`ENVIO10`	−10 %
K3	Inválida	cupón que no existe (`NOEXISTE`, `envio10`)	Error (F = 5 es impar)

3.2 Valores límite del peso

Con zona urbana, cliente normal, lunes y sin cupón, para que el costo dependa solo del peso. Se usa un paso de 0,01 kg.

Frontera	Valor	Clase	Esperado	Caso
0	0	P1 (inválida)	Error	CN-02
0	0,01	P2	$6.000	CN-03
L1 = 3	2,99	P2	$6.000	CN-04
L1 = 3	**3**	P2 - **el límite está incluido porque D = 0 es par**	$6.000	CN-05
L1 = 3	3,01	P3	$9.000	CN-06
L2 = 10	9,99	P3	$9.000	CN-07
L2 = 10	10	P3 (incluido)	$9.000	CN-08
L2 = 10	10,01	P4	$9.008 → $9.000	CN-09
PMAX = 45	44,99	P4	$36.992 → $37.000	CN-11
PMAX = 45	45	P4 (incluido)	$37.000	CN-12
PMAX = 45	45,01	P5 (inválida)	Error	CN-13

También se prueban los límites del día: 0 (error), 1, 5, 6, 7 y 8 (error) en los casos CN-18 a CN-23.

3.3 Tabla de decisión: cliente × cupón × fin de semana

Base de todos los casos: 5 kg, zona urbana → **$9.000**. "Fin de semana = sí" usa el domingo (día 7).

Por mis reglas, cuando es fin de semana los $2.000 se suman **antes** de los descuentos (E par), y cuando hay descuento de cliente y cupón se aplican **los dos**, primero cliente y luego cupón (B impar).

Regla	Cliente	Cupón	Fin de semana	+$2.000 antes	Desc. cliente	Desc. cupón	Cálculo	Esperado	Caso
1	normal	no	no	-	-	-	9.000	**$9.000**	CN-19
2	normal	no	sí	X	-	-	9.000 + 2.000	**$11.000**	CN-22
3	normal	ENVIO10	no	-	-	10 %	9.000 × 0,90	**$8.100**	CN-24
4	normal	ENVIO10	sí	X	-	10 %	11.000 × 0,90	**$9.900**	CN-29
5	frecuente	no	no	-	5 %	-	9.000 × 0,95 = 8.550	**$8.600**	CN-25
6	frecuente	no	sí	X	5 %	-	11.000 × 0,95 = 10.450	**$10.500**	CN-30
7	frecuente	ENVIO10	no	-	5 %	10 %	8.550 × 0,90 = 7.695	**$7.700**	CN-26
8	frecuente	ENVIO10	sí	X	5 %	10 %	10.450 × 0,90 = 9.405	**$9.400**	CN-31
9	vip	no	no	-	10 %	-	9.000 × 0,90	**$8.100**	CN-27
10	vip	no	sí	X	10 %	-	11.000 × 0,90	**$9.900**	CN-32
11	vip	ENVIO10	no	-	10 %	10 %	8.100 × 0,90 = 7.290	**$7.300**	CN-28
12	vip	ENVIO10	sí	X	10 %	10 %	9.900 × 0,90 = 8.910	**$8.900**	CN-33
13	cualquiera	no existe	cualquiera	-	-	-	-	**Error**	CN-34, CN-35

Por qué esta tabla distingue mis reglas de las de otro compañero:

- **Regla E.** En la regla 10, si el recargo fuera *después* de los descuentos, el resultado sería 9.000 × 0,90 + 2.000 = $10.100 y no $9.900.
- **Regla B.** En la regla 11, si solo se aplicara el *mayor* descuento, el resultado sería $8.100 y no $7.300.
- **Regla F.** En la regla 13, si el cupón inexistente se *ignorara*, el resultado sería un costo y no un error.

3.4 Matriz de casos de prueba

Técnicas: **PE** = partición de equivalencia, **VL** = valor límite, **TD** = tabla de decisión, **CE** = combinación de reglas (caso de extremo a extremo). Días: 1 = lunes, 5 = viernes, 6 = sábado, 7 = domingo.

ID	Requisito	Peso	Zona	Cliente	Día	Cupón	Resultado esperado	Técnica
CN-01	R1	−1	urbana	normal	1	-	Error	PE (P1)
CN-02	R1	0	urbana	normal	1	-	Error	VL (0)
CN-03	R1, R3	0,01	urbana	normal	1	-	$6.000	VL (0)
CN-04	R3	2,99	urbana	normal	1	-	$6.000	VL (L1)
CN-05	R3	3	urbana	normal	1	-	$6.000	VL (L1 incluido)
CN-06	R3	3,01	urbana	normal	1	-	$9.000	VL (L1)
CN-07	R3	9,99	urbana	normal	1	-	$9.000	VL (L2)
CN-08	R3	10	urbana	normal	1	-	$9.000	VL (L2)
CN-09	R3, R8	10,01	urbana	normal	1	-	$9.000	VL (L2)
CN-10	R3	20	urbana	normal	1	-	$17.000	PE (P4)
CN-11	R3, R8	44,99	urbana	normal	1	-	$37.000	VL (PMAX)
CN-12	R1, R3	45	urbana	normal	1	-	$37.000	VL (PMAX)
CN-13	R1	45,01	urbana	normal	1	-	Error	VL (PMAX)
CN-14	R1	50	urbana	normal	1	-	Error	PE (P5)
CN-15	R4	5	rural	normal	1	-	$10.800	PE (Z2)
CN-16	R4	5	nacional	normal	1	-	$13.500	PE (Z3)
CN-17	R2	5	internacional	normal	1	-	Error	PE (Z4)
CN-18	R2	5	urbana	normal	0	-	Error	VL (día)
CN-19	R2, R5	5	urbana	normal	1	-	$9.000	VL (día), TD-1
CN-20	R5	5	urbana	normal	5	-	$9.000	VL (día)
CN-21	R5	5	urbana	normal	6	-	$11.000	VL (día)
CN-22	R5	5	urbana	normal	7	-	$11.000	VL (día), TD-2
CN-23	R2	5	urbana	normal	8	-	Error	VL (día)
CN-24	R6	5	urbana	normal	1	ENVIO10	$8.100	TD-3
CN-25	R6, R8	5	urbana	frecuente	1	-	$8.600	TD-5
CN-26	R6, R8	5	urbana	frecuente	1	ENVIO10	$7.700	TD-7
CN-27	R6	5	urbana	vip	1	-	$8.100	TD-9
CN-28	R6, R8	5	urbana	vip	1	ENVIO10	$7.300	TD-11
CN-29	R5, R6	5	urbana	normal	7	ENVIO10	$9.900	TD-4
CN-30	R5, R6, R8	5	urbana	frecuente	7	-	$10.500	TD-6
CN-31	R5, R6, R8	5	urbana	frecuente	7	ENVIO10	$9.400	TD-8
CN-32	R5, R6	5	urbana	vip	7	-	$9.900	TD-10
CN-33	R5, R6, R8	5	urbana	vip	7	ENVIO10	$8.900	TD-12
CN-34	R7	5	urbana	normal	1	NOEXISTE	Error	TD-13, PE (K3)
CN-35	R7	5	urbana	vip	7	envio10	Error	TD-13, PE (K3)

ID	Requisito	Peso	Zona	Cliente	Día	Cupón	Resultado esperado	Técnica
CN-36	R8	2	urbana	vip	1	ENVIO10	$5.000	VL (costo mínimo)
CN-37	R6, R8	2	urbana	frecuente	1	ENVIO10	$5.100	VL (costo mínimo)
CN-38	R3 a R8	4	rural	vip	6	ENVIO10	$10.400	CE
CN-39	R3 a R8	12,5	nacional	vip	6	ENVIO10	$15.000	CE
CN-40	R3 a R8	45	nacional	frecuente	7	-	$54.600	CE
CN-41	R3 a R8	3	rural	frecuente	6	-	$8.700	CE
CN-42	R1	0	urbana	normal	1	-	Mensaje exacto "peso inválido"	PE (P1)

Son **42 casos**. Están automatizados en `tests/test_caja_negra.py` con el mismo ID.

3.5 Cálculo paso a paso

CN-38 - el ejemplo del enunciado con mis valores

Entradas: peso = 4 kg · zona = rural · cliente = vip · día = 6 (sábado) · cupón = ENVIO10

1. Tramo: D = 0 es par, el tramo 1 es peso ≤ 3. Como 4 no es ≤ 3 y sí es ≤ 10, va al tramo 2 → **$9.000**
2. Zona rural +20 % → 9.000 × 1,20 = **$10.800**
3. Sábado, y E = 6 es par: el recargo va antes de los descuentos → 10.800 + 2.000 = **$12.800**
4. B = 1 es impar: se aplican ambos. Primero VIP 10 % → 12.800 × 0,90 = **$11.520**; luego cupón 10 % → 11.520 × 0,90 = **$10.368**
5. Es mayor que $5.000. Se redondea a $100: 10.368 está más cerca de 10.400 → resultado esperado: **$10.400**

CN-39 - tramo 3 con decimales, zona nacional

Entradas: peso = 12,5 kg · zona = nacional · cliente = vip · día = 6 · cupón = ENVIO10

1. Tramo 3 (12,5 > 10): 9.000 + (12,5 − 10) × 800 = 9.000 + 2.000 = **$11.000**
2. Zona nacional +50 % → 11.000 × 1,50 = **$16.500**
3. Sábado, antes de los descuentos → 16.500 + 2.000 = **$18.500**
4. VIP 10 % → 18.500 × 0,90 = **$16.650**; cupón 10 % → 16.650 × 0,90 = **$14.985**
5. Mayor que $5.000. Redondeo: 14.985 → resultado esperado: **$15.000**

CN-40 - peso máximo, cliente frecuente, domingo

Entradas: peso = 45 kg · zona = nacional · cliente = frecuente · día = 7 · sin cupón

1. Peso válido (45 ≤ 45). Tramo 3: 9.000 + (45 − 10) × 800 = 9.000 + 28.000 = **$37.000**
2. Zona nacional +50 % → 37.000 × 1,50 = **$55.500**
3. Domingo, antes de los descuentos → 55.500 + 2.000 = **$57.500**
4. Frecuente 5 % → 57.500 × 0,95 = **$54.625**. No hay cupón.
5. Redondeo: 54.625 está más cerca de 54.600 que de 54.700 → resultado esperado: **$54.600**

CN-41 - justo en el límite L1, sábado

Entradas: peso = 3 kg · zona = rural · cliente = frecuente · día = 6 · sin cupón

1. Tramo: 3 ≤ 3, y el límite está incluido (D par) → tramo 1: **$6.000**
2. Zona rural +20 % → 6.000 × 1,20 = **$7.200**
3. Sábado, antes de los descuentos → 7.200 + 2.000 = **$9.200**
4. Frecuente 5 % → 9.200 × 0,95 = **$8.740**
5. Redondeo: 8.740 → resultado esperado: **$8.700**

CN-36 - costo mínimo

Entradas: peso = 2 kg · zona = urbana · cliente = vip · día = 1 (lunes) · cupón = ENVIO10

1. Tramo 1 (2 ≤ 3) → **$6.000**
2. Zona urbana: sin recargo → **$6.000**
3. Lunes: sin recargo de fin de semana.
4. VIP 10 % → 6.000 × 0,90 = **$5.400**; cupón 10 % → 5.400 × 0,90 = **$4.860**
5. 4.860 es menor que el mínimo → resultado esperado: **$5.000**

CN-30 - redondeo de los $50 hacia arriba

Entradas: peso = 5 kg · zona = urbana · cliente = frecuente · día = 7 (domingo) · sin cupón

1. Tramo 2 → **$9.000**
2. Urbana: sin recargo.
3. Domingo, antes del descuento → 9.000 + 2.000 = **$11.000**
4. Frecuente 5 % → 11.000 × 0,95 = **$10.450**
5. Queda justo en $50: redondea hacia arriba → resultado esperado: **$10.500**

CN-11 - justo debajo del peso máximo

Entradas: peso = 44,99 kg · zona = urbana · cliente = normal · día = 1 · sin cupón

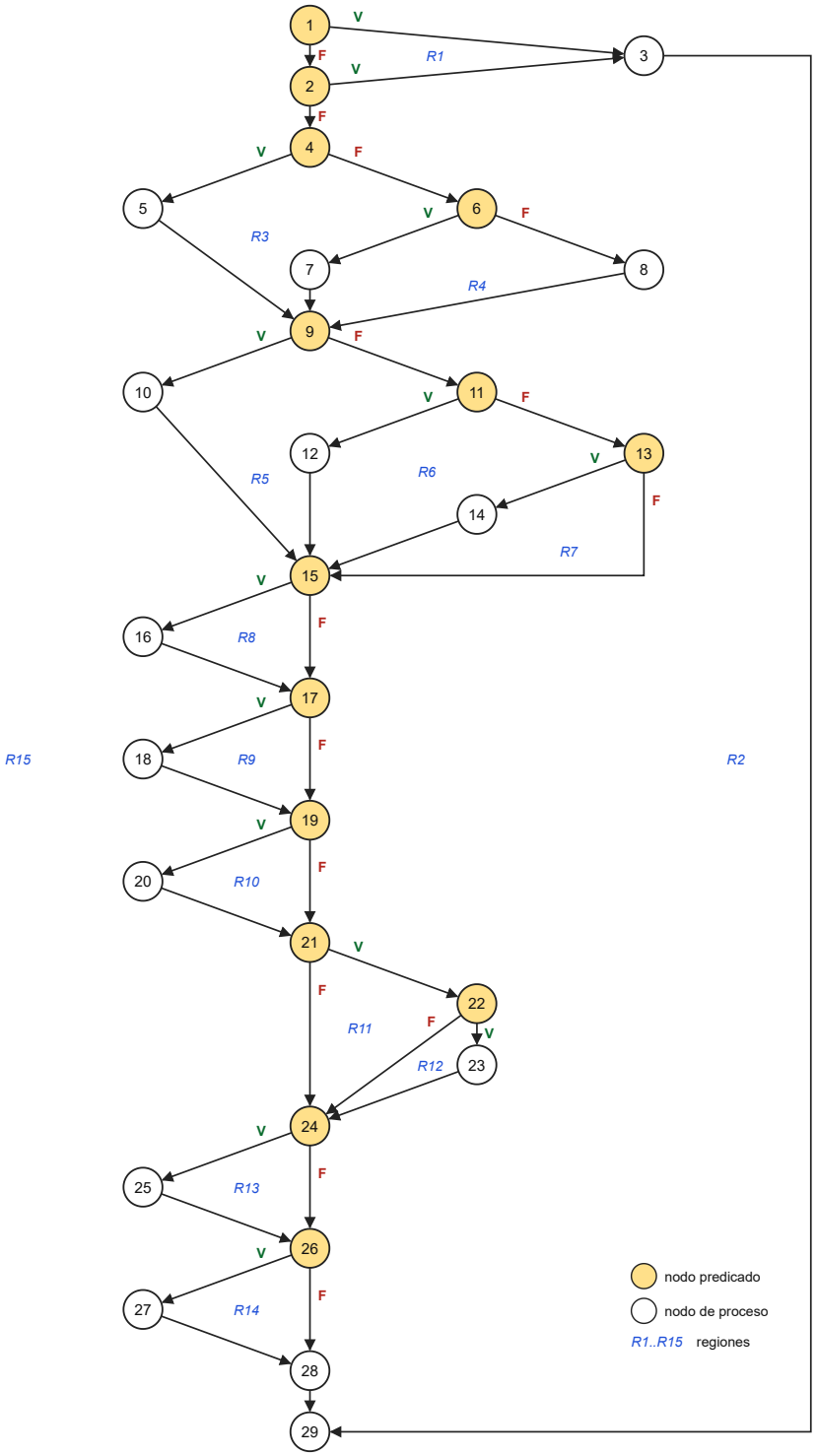
1. Peso válido (44,99 ≤ 45). Tramo 3: 9.000 + (44,99 − 10) × 800 = 9.000 + 34,99 × 800 = 9.000 + 27.992 = **$36.992**
2. Urbana, lunes, cliente normal, sin cupón: no cambia.
3. Redondeo: 36.992 está más cerca de 37.000 → resultado esperado: **$37.000**

Parte 4 - Caja blanca

Se analiza la función `calc` de la versión heredada 1.0.0 (`envios.py`, líneas 13 a 42).

4.1 Grafo de flujo

La condición compuesta de la línea 14 (`p <= 0 or p > 50`) se separa en dos nodos predicado (1 y 2), porque el `or` evalúa en cortocircuito y cada condición simple es una decisión.



Nodo	Línea	Sentencia	Tipo
1	14	`p <= 0`	Predicado
2	14	`p > 50`	Predicado

Nodo	Línea	Sentencia	Tipo
3	15	`raise ValueError("peso invalido")`	Proceso
4	16	`if p <= CONFIG["L1"]`	Predicado
5	17	`t = 6000`	Proceso
6	18	`elif p <= CONFIG["L2"]`	Predicado
7	19	`t = 9000`	Proceso
8	21	`t = 9000 + (p - CONFIG["L2"]) * 800`	Proceso
9	22	`if z == "urbana"`	Predicado
10	23	`t = t`	Proceso
11	24	`elif z == "rural"`	Predicado
12	25	`t = t + t * CONFIG["RECARGO_RURAL"]`	Proceso
13	26	`elif z == "nacional"`	Predicado
14	27	`t = t + t * CONFIG["RECARGO_NACIONAL"]`	Proceso
15	29	`if d > 6`	Predicado
16	30	`t = t + CONFIG["RECARGO_FINDE"]`	Proceso
17	31	`if c == "frecuente"`	Predicado
18	32	`t = t - t * 0.05`	Proceso
19	33	`if c == "vip"`	Predicado
20	34	`t = t - t * CONFIG["DESC_VIP"]`	Proceso
21	35	`if cup != None`	Predicado
22	36	`if cup == "ENVIO10"`	Predicado
23	37	`t = t - t * 0.10`	Proceso
24	38	`if t < 5000`	Predicado
25	39	`t = 5000`	Proceso
26	40	`if flag`	Predicado
27	41	`print("costo:", t)`	Proceso
28	42	`return int(t / 100) * 100`	Proceso
29	-	Fin (salida única: llegan el `raise` y el `return`)	Proceso

4.2 Complejidad ciclomática

a) Con aristas y nodos: V(G) = E − N + 2

- Aristas: los 14 nodos predicado tienen 2 salidas cada uno (28 aristas) y los 14 nodos de proceso que no son el final tienen 1 salida (14 aristas). E = 28 + 14 = **42**
- Nodos: N = **29**
- V(G) = 42 − 29 + 2 = **15**

b) Con nodos predicado: V(G) = P + 1

- Nodos predicado: 1, 2, 4, 6, 9, 11, 13, 15, 17, 19, 21, 22, 24 y 26 → P = **14**
- V(G) = 14 + 1 = **15**

c) Con regiones: V(G) = número de regiones del grafo

- Regiones cerradas: R1 a R14 (marcadas en el grafo).
- Región exterior: R15.
- V(G) = 14 + 1 = **15**

Las tres formas dan **V(G) = 15**. La herramienta `radon` confirma el valor: `calc - C (15)`. Es una complejidad alta para una sola función (más de 10 ya se considera difícil de probar y mantener), y se reduce en la Parte 6.

4.3 Caminos independientes y casos de prueba

Como V(G) = 15, el conjunto básico tiene 15 caminos. El camino 3 es el camino base; cada uno de los demás recorre al menos una arista nueva.

#	Camino (nodos)	Entradas `calc(p, z, c, d, cup, flag)`	Esperado según la especificación	Caso
1	1-3-29	`0, "urbana", "normal", 1`	Error	CB-01
2	1-2-3-29	`51, "urbana", "normal", 1`	Error	CB-02
3	1-2-4-6-7-9-10-15-17-19-21-24-26-28-29	`5, "urbana", "normal", 1`	$9.000	CB-03
4	1-2-4-**5**-9-10-15-17-19-21-24-26-28-29	`2, "urbana", "normal", 1`	$6.000	CB-04
5	1-2-4-6-**8**-9-10-15-17-19-21-24-26-28-29	`20, "urbana", "normal", 1`	$17.000	CB-05
6	1-2-4-6-7-9-**11-12**-15-17-19-21-24-26-28-29	`5, "rural", "normal", 1`	$10.800	CB-06
7	1-2-4-6-7-9-11-**13-14**-15-17-19-21-24-26-28-29	`5, "nacional", "normal", 1`	$13.500	CB-07
8	1-2-4-6-7-9-11-**13-15**-17-19-21-24-26-28-29	`5, "internacional", "normal", 1`	Error (R2)	CB-08
9	1-2-4-6-7-9-10-15-**16**-17-19-21-24-26-28-29	`5, "urbana", "normal", 7`	$11.000	CB-09
10	1-2-4-6-7-9-10-15-17-**18**-19-21-24-26-28-29	`5, "urbana", "frecuente", 1`	$8.600	CB-10
11	1-2-4-6-7-9-10-15-17-19-**20**-21-24-26-28-29	`5, "urbana", "vip", 1`	$8.100	CB-11
12	1-2-4-6-7-9-10-15-17-19-21-**22-23**-24-26-28-29	`5, "urbana", "normal", 1, "ENVIO10"`	$8.100	CB-12
13	1-2-4-6-7-9-10-15-17-19-21-**22-24**-26-28-29	`5, "urbana", "normal", 1, "OTRO"`	Error (R7)	CB-13
14	1-2-4-5-9-10-15-17-19-20-21-22-23-24-**25**-26-28-29	`2, "urbana", "vip", 1, "ENVIO10"`	$5.000	CB-14
15	1-2-4-6-7-9-10-15-17-19-21-24-26-**27**-28-29	`5, "urbana", "normal", 1, None, True`	$9.000 e imprime `costo:`	CB-15

Notas:

- El camino 14 necesita cambiar varias decisiones a la vez: la única forma de que el costo baje de $5.000 es tramo 1 con VIP y cupón (6.000 × 0,90 × 0,90 = 4.860).
- En los caminos 8 y 13 el **código** devuelve un valor, pero la **especificación** exige error. Esos dos casos fallan contra la versión 1.0.0 y destapan los defectos DEF-02 y DEF-05.
- El camino 10 pasa por la sentencia del descuento, pero el resultado falla por el redondeo (DEF-06): recorrer un camino no garantiza que el resultado sea correcto.

Los 15 casos están en `tests/test_caja_blanca.py`.

4.4 Cobertura de sentencias y de ramas

Cobertura de sentencias (100 %). Se logra con 9 casos: CB-01, CB-03, CB-05, CB-06, CB-07, CB-09, CB-10, CB-14 y CB-15.

Sentencia	Caso que la ejecuta
`raise` (nodo 3)	CB-01
`t = 9000` (7), `t = t` (10), `return` (28)	CB-03
tramo 3 (8)	CB-05
recargo rural (12)	CB-06
recargo nacional (14)	CB-07
recargo de fin de semana (16)	CB-09
descuento frecuente (18)	CB-10
`t = 6000` (5), descuento VIP (20), descuento de cupón (23), `t = 5000` (25)	CB-14
`print` (27)	CB-15

CB-04, CB-11 y CB-12 no hacen falta para sentencias porque CB-14 ya ejecuta el tramo 1, el descuento VIP y el descuento del cupón.

Medido con solo esos 9 casos: 29 de 29 sentencias, pero quedan 2 ramas sin recorrer (`26->29` y `36->38` en el reporte de cobertura).

Cobertura de ramas (100 %). A los 9 casos anteriores hay que sumar los que toman la salida *falsa* de dos decisiones que no tienen `else`:

Rama que falta	Línea	Caso
`z == "nacional"` falso (13 → 15)	26	CB-08
`cup == "ENVIO10"` falso (22 → 24)	36	CB-13

Con esos **11 casos** (CB-01, CB-03, CB-05 a CB-10, CB-13, CB-14 y CB-15) quedan cubiertas las 26 ramas que mide `coverage.py`.

Cobertura de condiciones. `coverage.py` cuenta el `if` de la línea 14 como una sola decisión, así que CB-01 basta para su rama verdadera. CB-02 (`p > 50` verdadero con `p <= 0` falso) se necesita para cubrir la segunda condición del `or`, que es justamente donde está el defecto DEF-01.

Resultado medido con `pytest --cov=envios --cov-branch` sobre la versión 1.0.0 con la suite completa: **29 de 29 sentencias y 26 de 26 ramas (100 %)**. Aun con cobertura total, 20 pruebas fallan: la cobertura dice qué código se ejecutó, no que el código haga lo que pide la especificación.

Parte 5 - Automatización, ejecución y defectos

5.1 Automatización

Todos los casos están automatizados con **pytest**:

Archivo	Casos	Pruebas
`tests/test_caja_negra.py`	CN-01 a CN-42	42
`tests/test_caja_blanca.py`	CB-01 a CB-15	15
Total		**57**

Cada prueba lleva el ID del caso, de modo que un fallo se rastrea directo a la matriz de la Parte 3 o a los caminos de la Parte 4.

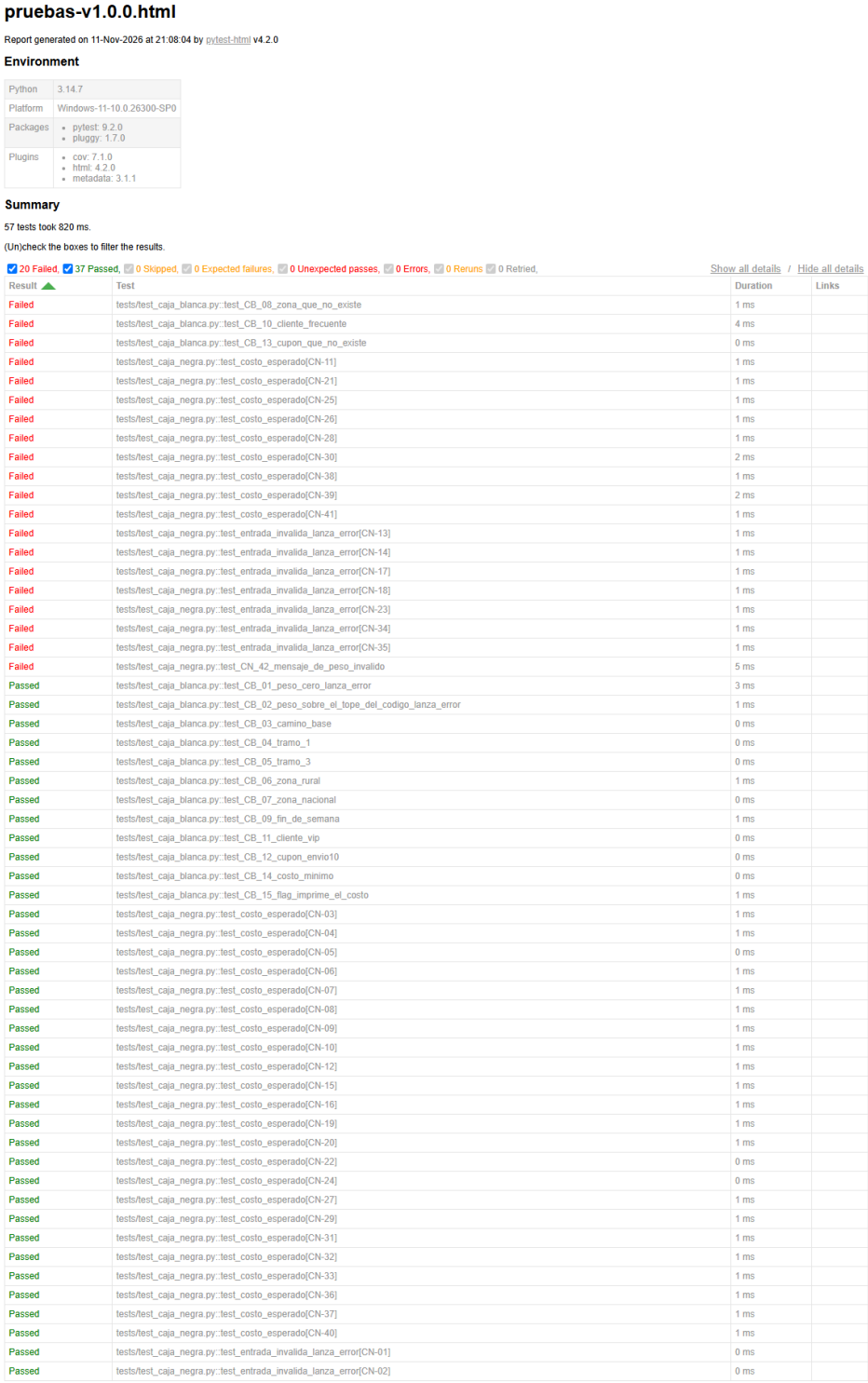
Ejecución contra el código heredado (etiqueta `v1.0.0`, sin modificar):

```
python -m pytest -v --tb=line --cov=envios --cov-branch --cov-report=term-missing
```

Resultado: **20 fallan y 37 pasan**. La salida completa está en `evidencias/01-pruebas-v1.0.0-heredado.txt` y el detalle de cada fallo en `evidencias/02-detalle-fallos-v1.0.0.txt`.

```
FAILED tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_blanca.py::test_CB_10_cliente_frecuente - assert 8500 == 8600
FAILED tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-11] - assert 36900 == 37000
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-21] - assert 9000 == 11000
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-25] - assert 8500 == 8600
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-26] - assert 7600 == 7700
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-28] - assert 7200 == 7300
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-30] - assert 10400 == 10500
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-38] - assert 8700 == 10400
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-39] - assert 13300 == 15000
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-41] - assert 6800 == 8700
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-13] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-14] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-17] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-18] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-23] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-34] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-35] - Failed: DID NOT RAISE ValueError
FAILED tests/test_caja_negra.py::test_CN_42_mensaje_de_peso_invalido - assert 'peso invalido' == 'peso inválido'
====================== 20 failed, 37 passed in 0.59s ======================
```

Captura del reporte de pytest (`--html`, plugin pytest-html) de esa misma ejecución: 20 pruebas en rojo y 37 en verde.



pruebas-v1.0.0.html

Report generated on 11-Nov-2026 at 21:08:04 by pytest-html v4.2.0

Environment

Python	3.14.7
Platform	Windows-11-10.0.26300-SP0
Packages	• pytest: 9.2.0 • pluggy: 1.7.0
Plugins	• cov: 7.1.0 • html: 4.2.0 • metadata: 3.1.1

Summary

57 tests took 820 ms.

(Un)check the boxes to filter the results.

20 Failed, 37 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Failed	tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe	1 ms	
Failed	tests/test_caja_blanca.py::test_CB_10_cliente_frecuente	4 ms	
Failed	tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe	0 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-11]	1 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-21]	1 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-25]	1 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-26]	1 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-28]	1 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-30]	2 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-38]	1 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-39]	2 ms	
Failed	tests/test_caja_negra.py::test_costo_esperado[CN-41]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-13]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-14]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-17]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-18]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-23]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-34]	1 ms	
Failed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-35]	1 ms	
Failed	tests/test_caja_negra.py::test_CN_42_mensaje_de_peso_invalido	5 ms	
Passed	tests/test_caja_blanca.py::test_CB_01_peso_cero_lanza_error	3 ms	
Passed	tests/test_caja_blanca.py::test_CB_02_peso_sobre_el_tope_del_codigo_lanza_error	1 ms	
Passed	tests/test_caja_blanca.py::test_CB_03_camino_base	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_04_tramo_1	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_05_tramo_3	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_06_zona_rural	1 ms	
Passed	tests/test_caja_blanca.py::test_CB_07_zona_nacional	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_09_fin_de_semana	1 ms	
Passed	tests/test_caja_blanca.py::test_CB_11_cliente_vip	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_12_cupon_envio10	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_14_costo_minimo	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_15_flag_imprime_el_costo	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-03]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-04]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-05]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-06]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-07]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-08]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-09]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-10]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-12]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-15]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-16]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-19]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-20]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-22]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-24]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-27]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-29]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-31]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-32]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-33]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-36]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-37]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-40]	1 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-01]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-02]	0 ms	

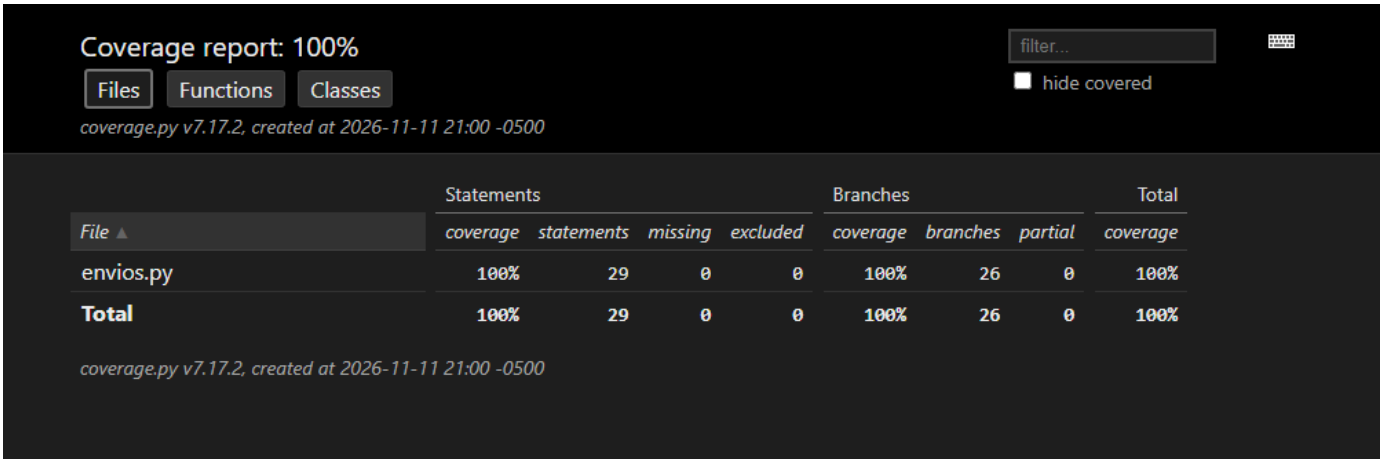
5.2 Cobertura

Medida con **pytest-cov** (coverage.py) con cobertura de ramas:

```
Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
envios.py      29      0     26      0   100%
-----------------------------------------------------
TOTAL          29      0     26      0   100%
```

- Cobertura de sentencias: **29 / 29 = 100 %**
- Cobertura de ramas: **26 / 26 = 100 %**

Capturas del reporte HTML de coverage.py (`python -m pytest --cov=envios --cov-branch --cov-report=html`) sobre la versión 1.0.0:



Coverage report: 100%

filter...

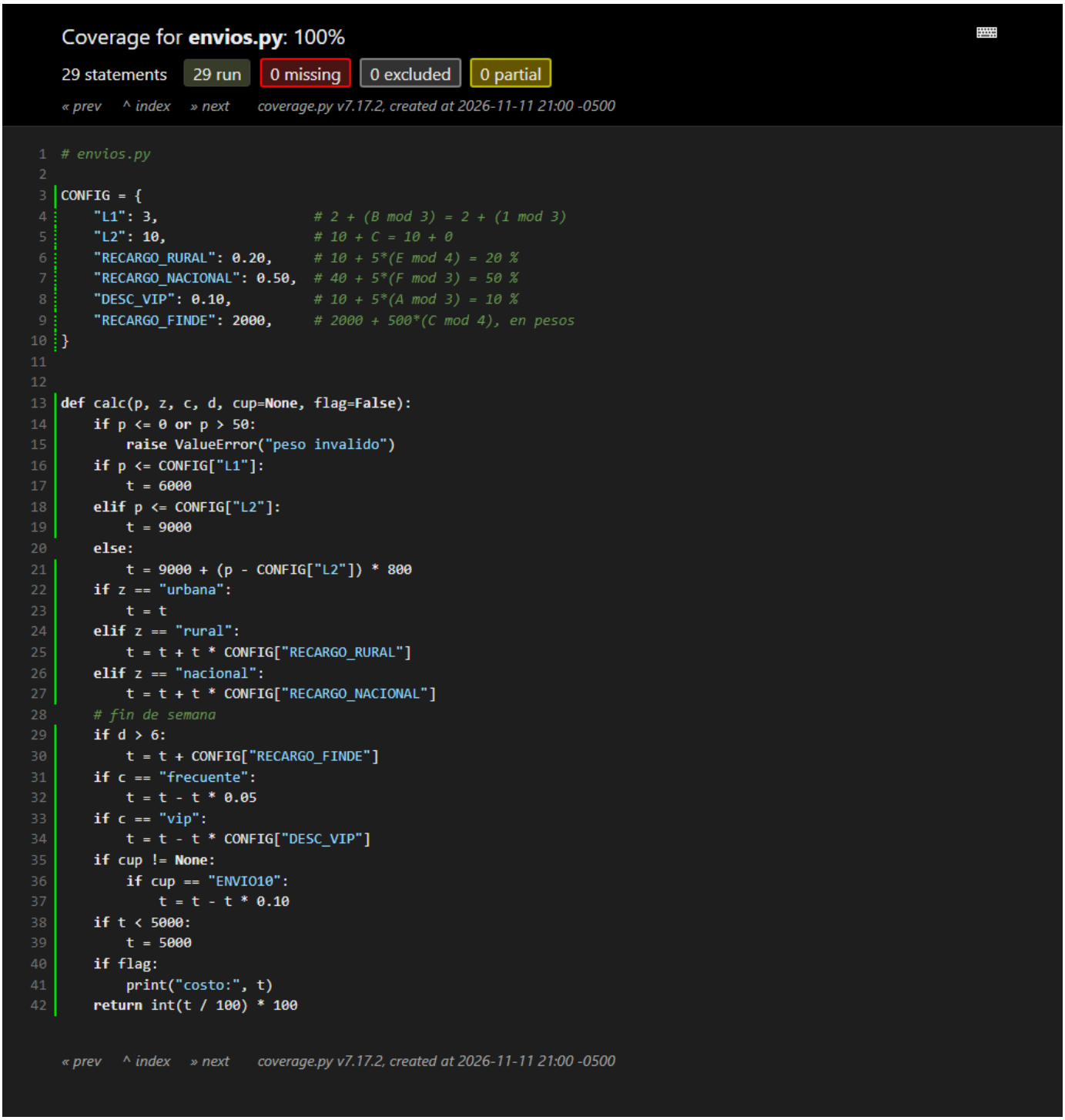
hide covered

Files | Functions | Classes

coverage.py v7.17.2, created at 2026-11-11 21:00 -0500

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
envios.py	100%	29	0	0	100%	26	0	100%
Total	100%	29	0	0	100%	26	0	100%

coverage.py v7.17.2, created at 2026-11-11 21:00 -0500



Coverage for **envios.py**: 100%

29 statements | 29 run | 0 missing | 0 excluded | 0 partial

« prev ^ index » next coverage.py v7.17.2, created at 2026-11-11 21:00 -0500

```python
# envios.py

CONFIG = {
    "L1": 3,                  # 2 + (B mod 3) = 2 + (1 mod 3)
    "L2": 10,                 # 10 + C = 10 + 0
    "RECARGO_RURAL": 0.20,    # 10 + 5*(E mod 4) = 20 %
    "RECARGO_NACIONAL": 0.50, # 40 + 5*(F mod 3) = 50 %
    "DESC_VIP": 0.10,         # 10 + 5*(A mod 3) = 10 %
    "RECARGO_FINDE": 2000,    # 2000 + 500*(C mod 4), en pesos
}


def calc(p, z, c, d, cup=None, flag=False):
    if p <= 0 or p > 50:
        raise ValueError("peso invalido")
    if p <= CONFIG["L1"]:
        t = 6000
    elif p <= CONFIG["L2"]:
        t = 9000
    else:
        t = 9000 + (p - CONFIG["L2"]) * 800
    if z == "urbana":
        t = t
    elif z == "rural":
        t = t + t * CONFIG["RECARGO_RURAL"]
    elif z == "nacional":
        t = t + t * CONFIG["RECARGO_NACIONAL"]
    # fin de semana
    if d > 6:
        t = t + CONFIG["RECARGO_FINDE"]
    if c == "frecuente":
        t = t - t * 0.05
    if c == "vip":
        t = t - t * CONFIG["DESC_VIP"]
    if cup != None:
        if cup == "ENVIO10":
            t = t - t * 0.10
    if t < 5000:
        t = 5000
    if flag:
        print("costo:", t)
    return int(t / 100) * 100
```

« prev ^ index » next coverage.py v7.17.2, created at 2026-11-11 21:00 -0500

Todas las líneas quedan marcadas en verde (ejecutadas) y no hay ramas parciales.

5.3 Registro de defectos

Qué líneas del código heredado **son** defecto y cuáles **no** depende de mi especificación:

Línea	Código	¿Defecto para mi código?
14	`p > 50`	**Sí** - mi PMAX es 45 (DEF-01)
15	`"peso invalido"`	**Sí** - la especificación pide "peso inválido" (DEF-07)
16	`p <= CONFIG["L1"]`	No - D = 0 es par, el límite sí se incluye
22-27	zonas sin `else`	**Sí** - una zona inválida no da error (DEF-02)
29	`d > 6`	**Sí** - deja por fuera el sábado (DEF-04) y no valida el día (DEF-03)
29-30	recargo antes de los descuentos	No - E = 6 es par, va antes
31-37	se aplican los dos descuentos	No - B = 1 es impar, se aplican ambos
35-37	cupón desconocido se ignora	**Sí** - F = 5 es impar, debe ser error (DEF-05)
42	`int(t / 100) * 100`	**Sí** - trunca en vez de redondear (DEF-06)

DEF-01 - Acepta envíos de más de 45 kg

Campo	Detalle
Requisito que incumple	R1 (0 < peso ≤ 45)
Pasos para reproducir	1. Llenar CONFIG con mis valores. 2. Ejecutar `calc(45.01, "urbana", "normal", 1)`. 3. Ejecutar `calc(50, "urbana", "normal", 1)`.
Resultado esperado	En los dos casos, error "peso inválido".
Resultado obtenido	Devuelve $37.000 y $41.000: el envío se acepta y se cobra.
Causa	Línea 14: el tope está escrito a mano como `50` y no usa PMAX = 45.
Casos que lo detectan	CN-13, CN-14
Severidad	**Alta.** El sistema acepta paquetes que la empresa no está en capacidad de transportar; el envío se cobra y luego no se puede despachar.
Prioridad	**Alta.** Es una regla de operación básica y la corrección es de una línea.

DEF-02 - Una zona que no existe se cobra como urbana

Campo	Detalle
Requisito que incumple	R2 (zona inválida → error)
Pasos para reproducir	1. Ejecutar `calc(5, "internacional", "normal", 1)`.
Resultado esperado	Error por zona inválida.
Resultado obtenido	Devuelve $9.000, como si fuera urbana.
Causa	Líneas 22-27: la cadena `if / elif` no tiene `else`; cualquier texto pasa sin recargo.
Casos que lo detectan	CN-17, CB-08
Severidad	**Alta.** Un error de digitación ("Rural" con mayúscula, "nacinal") hace que no se cobre el recargo del 20 % o 50 %, y se aceptan destinos que la empresa no atiende.
Prioridad	**Media.** Normalmente la zona se escoge de una lista, así que ocurre menos que DEF-04 y DEF-06.

DEF-03 - No valida que el día esté entre 1 y 7

Campo	Detalle
Requisito que incumple	R2 (día fuera de 1 a 7 → error)
Pasos para reproducir	1. Ejecutar `calc(5, "urbana", "normal", 0)`. 2. Ejecutar `calc(5, "urbana", "normal", 8)`.
Resultado esperado	Error en los dos casos.
Resultado obtenido	Con día 0 devuelve $9.000. Con día 8 devuelve $11.000: además lo trata como fin de semana.
Causa	No existe ninguna validación del día; la única comparación es `d > 6` (línea 29).
Casos que lo detectan	CN-18, CN-23
Severidad	**Media.** Produce cobros con datos imposibles (día 8 cobra recargo), pero solo si llega un día inválido.
Prioridad	**Media.** Se corrige junto con las demás validaciones.

DEF-04 - El sábado no cobra el recargo de fin de semana

Campo	Detalle
Requisito que incumple	R5 (sábado y domingo: +$2.000)
Pasos para reproducir	1. Ejecutar `calc(5, "urbana", "normal", 6)`.
Resultado esperado	$11.000 (9.000 + 2.000).
Resultado obtenido	$9.000: no suma el recargo.
Causa	Línea 29: `d > 6` solo es verdadero el domingo (7). Debería ser `d >= 6`.
Casos que lo detectan	CN-21, CN-38, CN-39, CN-41
Severidad	**Crítica.** La empresa deja de cobrar $2.000 en **todos** los envíos de **todos** los sábados. Es pérdida de dinero directa y repetida.
Prioridad	**Urgente.** Es el primero que se debe corregir: cada sábado que pasa cuesta dinero y el arreglo es un solo carácter.

DEF-05 - Un cupón que no existe se ignora en silencio

Campo	Detalle
Requisito que incumple	R7 (para mi código, F = 5 impar: es error)
Pasos para reproducir	1. Ejecutar `calc(5, "urbana", "normal", 1, "NOEXISTE")`. 2. Ejecutar `calc(5, "urbana", "vip", 7, "envio10")`.
Resultado esperado	Error por cupón inválido en los dos casos.
Resultado obtenido	Devuelve $9.000 y $9.900: cobra sin descuento y sin avisar.
Causa	Líneas 35-37: si el cupón no es `ENVIO10` simplemente no hace nada.
Casos que lo detectan	CN-34, CN-35, CB-13
Severidad	**Media.** El cliente cree que usó un cupón y paga el valor completo sin enterarse; genera reclamos, pero la empresa no pierde dinero.
Prioridad	**Media.**

DEF-06 - El resultado se trunca en vez de redondearse

Campo	Detalle
Requisito que incumple	R8 (redondeo al múltiplo de $100 más cercano; los $50 suben)
Pasos para reproducir	1. Ejecutar `calc(5, "urbana", "frecuente", 1)`. 2. Ejecutar `calc(44.99, "urbana", "normal", 1)`.

Campo	Detalle
Resultado esperado	$8.600 (8.550 sube) y $37.000 (36.992 sube).
Resultado obtenido	$8.500 y $36.900.
Causa	Línea 42: `int(t / 100) * 100` corta los decimales, siempre hacia abajo.
Casos que lo detectan	CN-11, CN-25, CN-26, CN-28, CN-30, CB-10 (y también afecta a CN-38 y CN-39)
Severidad	**Alta.** Afecta a cualquier envío cuyo valor termine entre $50 y $99: se cobran hasta $99 de menos por envío. Es poco por envío pero ocurre todos los días.
Prioridad	**Alta.**

DEF-07 - El mensaje de error no coincide con la especificación

Campo	Detalle
Requisito que incumple	R1 (el error es "peso inválido")
Pasos para reproducir	1. Ejecutar `calc(0, "urbana", "normal", 1)` y leer el mensaje del error.
Resultado esperado	`ValueError` con el mensaje `peso inválido`.
Resultado obtenido	`ValueError` con el mensaje `peso invalido` (sin tilde).
Causa	Línea 15: texto mal escrito.
Casos que lo detectan	CN-42
Severidad	**Baja.** Es cosmético: el error sí se lanza. Solo afectaría a quien compare el texto exacto.
Prioridad	**Baja.** Se corrige porque es barato, al final.

Resumen

ID	Título	Requisito	Severidad	Prioridad	Casos que fallan
DEF-01	Acepta envíos de más de 45 kg	R1	Alta	Alta	2
DEF-02	Zona inexistente se cobra como urbana	R2	Alta	Media	2
DEF-03	No valida el día	R2	Media	Media	2
DEF-04	El sábado no cobra recargo	R5	Crítica	Urgente	4
DEF-05	Cupón inexistente se ignora	R7	Media	Media	3
DEF-06	Trunca en vez de redondear	R8	Alta	Alta	6
DEF-07	Mensaje de error sin tilde	R1	Baja	Baja	1
				Total	**20**

Defectos enmascarados. CN-38 y CN-39 fallan primero por DEF-04 (es sábado), pero aunque se corrija ese defecto seguirían fallando por DEF-06 (10.368 y 14.985 se truncarían). Por eso se vuelve a ejecutar toda la suite después de cada corrección.

Observaciones que no son defecto (la especificación no las cubre):

- Un tipo de cliente desconocido se acepta y se cobra sin descuento.
- El parámetro `flag` imprime en consola; es un mal olor, se trata en la Parte 6.

5.4 Métricas

Métrica	Cálculo	Valor
Casos ejecutados	42 de caja negra + 15 de caja blanca	57
Casos que pasan	37 / 57	**64,9 %**
Casos que fallan	20 / 57	35,1 %
Defectos encontrados		**7**

Métrica	Cálculo	Valor
Defectos por severidad	Crítica 1 · Alta 3 · Media 2 · Baja 1	7
Densidad de defectos (función `calc`, 30 líneas)	7 / 30 × 100	**23,3 defectos por cada 100 líneas**
Densidad de defectos (archivo completo, 42 líneas)	7 / 42 × 100	16,7 defectos por cada 100 líneas
Cobertura de sentencias	29 / 29	**100 %**
Cobertura de ramas	26 / 26	**100 %**
Complejidad ciclomática de `calc`	radon	15

Lectura: con el 100 % de cobertura todavía falla el 35 % de los casos. La cobertura sola no encuentra defectos de **omisión** (validaciones que faltan, como DEF-02, DEF-03 y DEF-05): no se puede cubrir código que no existe. Esos defectos solo los encuentran las pruebas de caja negra hechas contra la especificación.

Parte 6 - Mantenimiento y refactorización

6.1 Malos olores del código heredado

Las líneas son las de `envios.py` en la versión 1.0.0.

#	Mal olor (nombre técnico)	Línea(s)	Dónde se ve	Deuda técnica que genera
1	**Números mágicos** (Magic Numbers)	14, 17, 19, 21, 29, 32, 37, 38, 39, 42	`50`, `6000`, `9000`, `800`, `6`, `0.05`, `0.10`, `5000`, `100` escritos directo en el código	Nadie sabe qué significa cada número ni cuáles dependen de la tarifa. El `50` de la línea 14 quedó desactualizado frente al peso máximo real y produjo DEF-01. Cambiar una tarifa obliga a buscar el número por todo el código.
2	**Método largo** (Long Method) con varias responsabilidades	13-42	Una sola función valida, calcula la tarifa, recarga por zona y fin de semana, descuenta, aplica el mínimo, redondea e imprime	Complejidad ciclomática de 15. No se puede probar ni cambiar una regla sin tocar las demás; cada cambio arriesga todo el cálculo.
3	**Argumento bandera** (Flag Argument) con efecto secundario	13, 40-41	`flag=False` decide si la función además imprime	La función hace dos cosas (calcular e imprimir). Mezcla lógica de negocio con salida por consola y duplica los caminos a probar.
4	**Nombres poco comunicativos** (Mysterious Name)	13 y todo el cuerpo	`calc`, `p`, `z`, `c`, `d`, `cup`, `t`, `flag`	Hay que leer todo el cuerpo para saber qué es cada cosa; es fácil pasar los argumentos en el orden equivocado.
5	**Código muerto** (Dead Code)	22-23	`if z == "urbana": t = t`	Una asignación que no hace nada: confunde al lector y esconde que falta el `else` de zona inválida (DEF-02).
6	**Sentencias switch / condicionales sobre un tipo** (Switch Statements)	22-27 y 31-34	Cadenas `if / elif` que comparan `z` y `c` contra textos	Agregar una zona o un tipo de cliente obliga a modificar `calc` (viola abierto/cerrado). Es justo lo que resuelve el patrón Estrategia.
7	**Código duplicado** (Duplicated Code)	25, 27, 32, 34, 37	El mismo cálculo `t = t ± t * porcentaje` repetido cinco veces	Si cambia la forma de aplicar un porcentaje hay que corregir cinco lugares y es fácil olvidar uno.
8	**Obsesión por primitivos** (Primitive Obsession)	22-36	Zonas, clientes y cupones son textos sueltos sin lista de valores válidos	No hay validación: cualquier texto pasa. De ahí salen DEF-02 y DEF-05.
9	**Datos globales mutables** (Global Data)	3-10	`CONFIG` es un diccionario global con claves de texto	Cualquier parte del programa lo puede cambiar; un error en una clave solo se descubre al ejecutar (`KeyError`).
10	**Comentario como desodorante** (Comments)	28	`# fin de semana` sobre `if d > 6`	El comentario dice una cosa y el código hace otra (solo domingo): el comentario tapó DEF-04. El código debería explicarse solo.
11	**Dinero en punto flotante**	25-37, 42	Valores en pesos calculados con `float`	Errores de representación (por ejemplo 8.549,999... en vez de 8.550) que cambian el redondeo.

6.2 Clasificación de los cambios

Cambio	Tipo de mantenimiento	Por qué
Corregir DEF-01 a DEF-07	**Correctivo**	Arreglan fallas: el código no hacía lo que dice la especificación.
Reemplazar números mágicos por constantes con nombre	**Perfectivo**	Mejora la legibilidad y la facilidad de cambio sin alterar el resultado.
Extraer funciones y renombrar (`calcular_costo`, `peso`, `zona`...)	**Perfectivo**	Reduce la complejidad; el comportamiento es el mismo.

Cambio	Tipo de mantenimiento	Por qué
Eliminar el argumento `flag` y el `print`	**Perfectivo**	Quita una responsabilidad que no es del cálculo. Cambia la interfaz (ver Parte 7).
Eliminar `t = t`	**Perfectivo**	Limpieza de código muerto.
Patrón Estrategia para las zonas	**Perfectivo y preventivo**	Mejora el diseño y deja el sistema listo para zonas nuevas sin tocar `calcular_costo`.
Manejar el dinero con `Decimal`	**Preventivo**	Evita errores futuros de redondeo por punto flotante antes de que aparezcan.
Validar que el día sea un entero de 1 a 7 (no 6,5)	**Preventivo**	Cierra una entrada rara antes de que cause un cobro mal hecho.
Pruebas de unidad para cada función extraída	**Preventivo**	Red de seguridad para los cambios que vienen.
Hora pico (Parte 8)	**Perfectivo**	Es una funcionalidad nueva que pide el negocio.
(ninguno)	**Adaptativo**	No hubo cambios de entorno: mismo lenguaje, misma plataforma. Sería adaptativo, por ejemplo, migrar a otra versión de Python o a otro sistema de facturación.

6.3 Refactorización

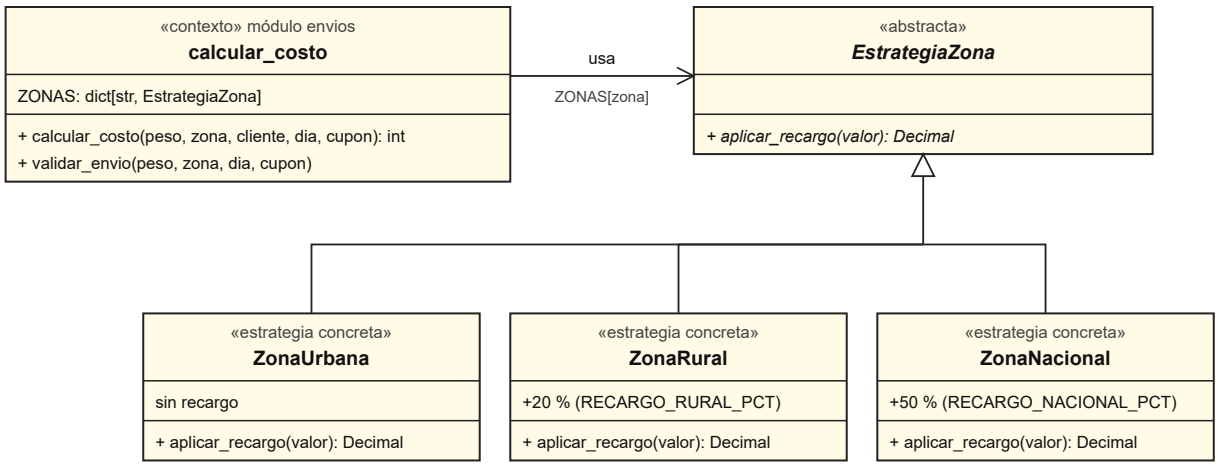
Se hizo en la rama `refactor/estrategia`, **después** de corregir los defectos y con la suite en verde, en dos pasos pequeños:

Paso 1 - Extraer funciones, constantes con nombre y eliminar `flag`.

Antes (dentro de `calc`)	Después
Validaciones repartidas por el cuerpo	`validar_envio(peso, zona, dia, cupon)`
`if p <= L1 … elif … else`	`tarifa_base(peso)`
`if d >= 6: t = t + …`	`recargo_fin_de_semana(dia)`
Tres `if` de descuentos	`aplicar_descuentos(valor, cliente, cupon)`
`if t < 5000` … y `int(round(…))`	`redondear(valor)`
`t = t ± t * x` repetido	`sumar_porcentaje` y `restar_porcentaje`
`50`, `6000`, `9000`, `800`, `0.05`, `5000`, `100`…	`PESO_MAXIMO_KG`, `TARIFA_TRAMO_1`, `TARIFA_TRAMO_2`, `TARIFA_KG_ADICIONAL`, `DESCUENTO_FRECUENTE_PCT`, `COSTO_MINIMO`, `MULTIPLO_REDONDEO`…
`calc(p, z, c, d, cup=None, flag=False)`	`calcular_costo(peso, zona, cliente, dia, cupon=None)`; `calc` se conserva como alias
`flag` y `print("costo:", t)`	Eliminados: quien quiera ver el costo imprime el valor que devuelve la función
`t = t`	Eliminada
Dinero en `float`	`Decimal`, con redondeo `ROUND_HALF_UP`

Paso 2 - Patrón Estrategia para el recargo por zona.

La cadena `if / elif` de zonas se reemplazó por una familia de clases con la misma interfaz. `calcular_costo` ya no pregunta qué zona es: le pide a la estrategia de esa zona que aplique su recargo.




```
class EstrategiaZona(ABC):
    @abstractmethod
    def aplicar_recargo(self, valor): ...

class ZonaUrbana(EstrategiaZona):
    def aplicar_recargo(self, valor):
        return valor

class ZonaRural(EstrategiaZona):
    def aplicar_recargo(self, valor):
        return sumar_porcentaje(valor, RECARGO_RURAL_PCT)

class ZonaNacional(EstrategiaZona):
    def aplicar_recargo(self, valor):
        return sumar_porcentaje(valor, RECARGO_NACIONAL_PCT)

ZONAS = {"urbana": ZonaUrbana(), "rural": ZonaRural(), "nacional": ZonaNacional()}

# en calcular_costo:
valor = ZONAS[zona].aplicar_recargo(valor)
```

Participantes del patrón:

Rol	En el código
Estrategia (interfaz)	`EstrategiaZona`
Estrategias concretas	`ZonaUrbana`, `ZonaRural`, `ZonaNacional`
Contexto	`calcular_costo`, que escoge la estrategia con `ZONAS[zona]`

Ventaja: para agregar una zona se crea una clase nueva y se registra en `ZONAS`; `calcular_costo` y `validar_envio` no se tocan (la validación usa el mismo diccionario).

6.4 Corrección de los defectos

Cada defecto se corrigió en su propia rama, en orden de prioridad, con el cambio mínimo sobre el código heredado. Después de cada rama se ejecutó toda la suite:

Orden	Rama	Corrección	Fallan antes	Fallan después
1	`fix/DEF-04`	`d > 6` → `d >= 6`	20	18
2	`fix/DEF-06`	Redondeo al múltiplo de 100 más cercano en vez de truncar	18	10
3	`fix/DEF-01`	Tope de peso `50` → `CONFIG["PMAX"]` = 45	10	8
4	`fix/DEF-02`	`else: raise ValueError("zona inválida")`	8	6
5	`fix/DEF-03`	Validación `d < 1 or d > 7`	6	4
6	`fix/DEF-05`	`else: raise ValueError("cupón inválido")`	4	1
7	`fix/DEF-07`	Mensaje `"peso inválido"`	1	**0**

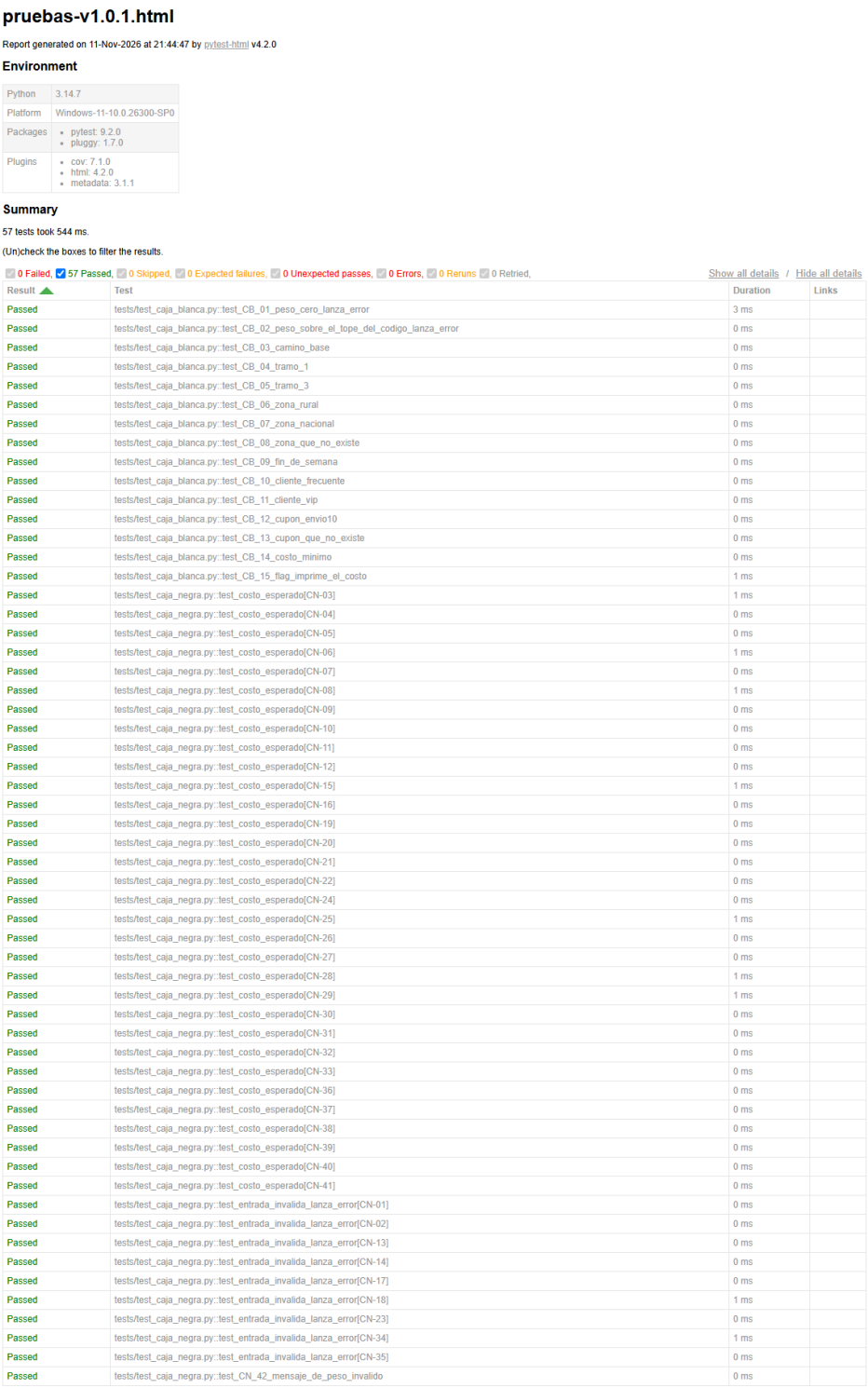
Al corregir DEF-04 solo se arreglaron 2 de sus 4 casos: CN-38 y CN-39 siguieron fallando hasta corregir DEF-06, como se había previsto en la Parte 5 (defectos enmascarados).

6.5 Pruebas de regresión

Se ejecutaron **las mismas 57 pruebas** de las Partes 3 y 4, sin cambiar ningún resultado esperado:

Versión	Pasan	Fallan
v1.0.0 (heredado)	37	20
v1.0.1 (defectos corregidos)	57	0
v2.0.0 (refactorizado)	57	0

Captura del reporte de pytest sobre la versión corregida (v1.0.1): las 57 pruebas en verde.



pruebas-v1.0.1.html

Report generated on 11-Nov-2026 at 21:44:47 by pytest-html v4.2.0

Environment

Python	3.14.7
Platform	Windows-11-10.0.26300-SP0
Packages	• pytest: 9.2.0 • pluggy: 1.7.0
Plugins	• cov: 7.1.0 • html: 4.2.0 • metadata: 3.1.1

Summary

57 tests took 544 ms.

(Un)check the boxes to filter the results.

0 Failed, 57 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_caja_blanca.py::test_CB_01_peso_cero_lanza_error	3 ms	
Passed	tests/test_caja_blanca.py::test_CB_02_peso_sobre_el_tope_del_codigo_lanza_error	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_03_camino_base	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_04_tramo_1	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_05_tramo_3	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_06_zona_rural	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_07_zona_nacional	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_09_fin_de_semana	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_10_cliente_frecuente	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_11_cliente_vip	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_12_cupon_envio10	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_14_costo_minimo	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_15_flag_imprime_el_costo	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-03]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-04]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-05]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-06]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-07]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-08]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-09]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-10]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-11]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-12]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-15]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-16]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-19]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-20]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-21]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-22]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-24]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-25]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-26]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-27]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-28]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-29]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-30]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-31]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-32]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-33]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-36]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-37]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-38]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-39]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-40]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-41]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-01]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-02]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-13]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-14]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-17]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-18]	1 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-23]	0 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-34]	1 ms	
Passed	tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-35]	0 ms	
Passed	tests/test_caja_negra.py::test_CN_42_mensaje_de_peso_invalido	0 ms	

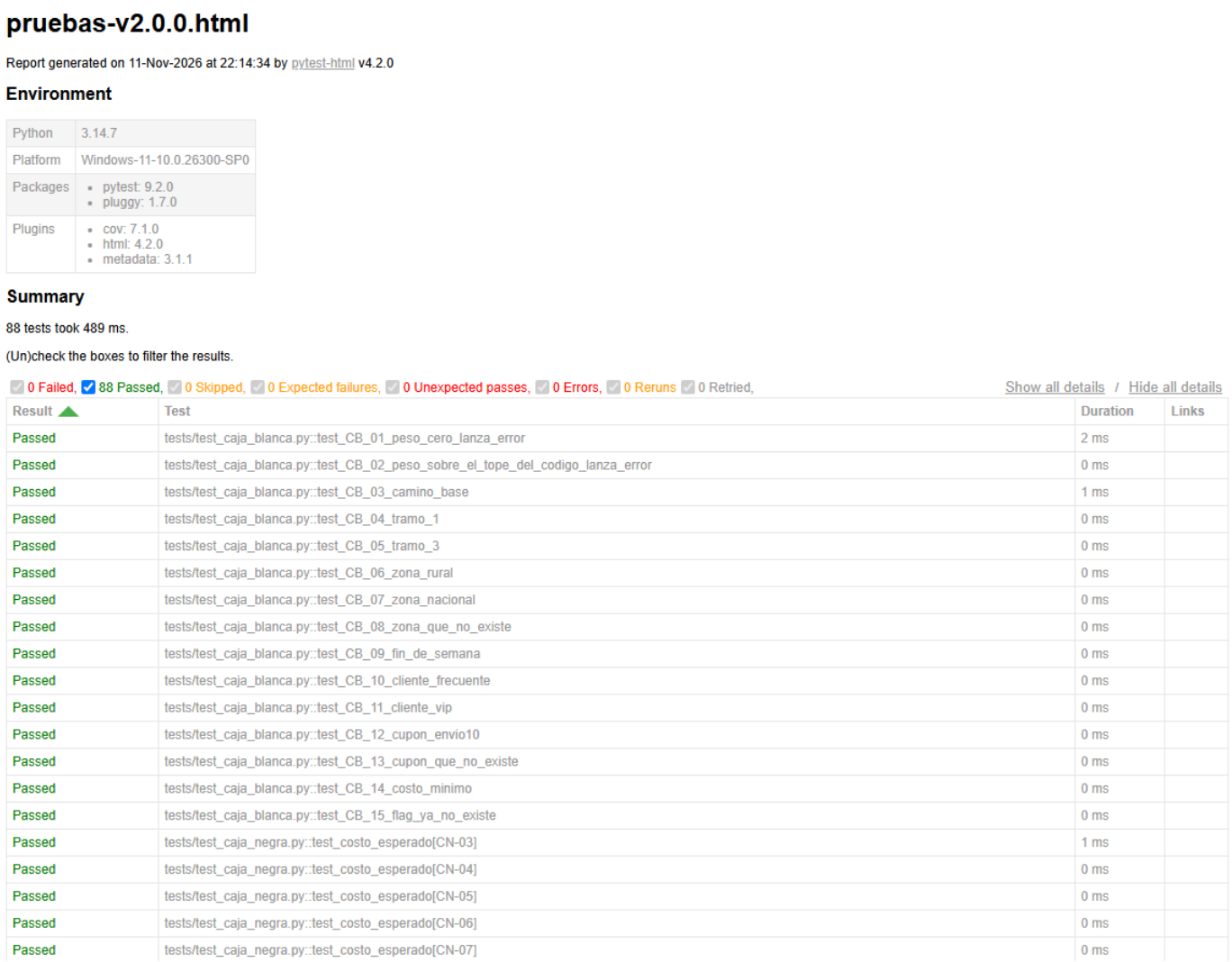
Único ajuste a una prueba: **CB-15** probaba el camino de `flag=True`. Como el parámetro se eliminó, ese camino ya no existe; la prueba ahora verifica que pasar un sexto argumento posicional produce `TypeError`. No es una prueba que se "acomodó" para pasar: es la consecuencia documentada del cambio de interfaz.

Además se agregaron 31 pruebas de unidad (`tests/test_unidades.py`) para las funciones extraídas y las estrategias. Total en v2.0.0: **88 pruebas, 88 pasan**.

```
$ python -m pytest -q --cov=envios --cov-branch --cov-report=term-missing
.................................................................... [ 81%]
................                                                     [100%]
Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
envios.py      71      0     12      0   100%
-----------------------------------------------------
TOTAL          71      0     12      0   100%
88 passed
```

La salida completa está en evidencias/04-regresion-v2.0.0.txt.

Capturas sobre la versión refactorizada (v2.0.0): reporte de pytest con las 88 pruebas en verde y reporte de cobertura.



pruebas-v2.0.0.html

Report generated on 11-Nov-2026 at 22:14:34 by pytest-html v4.2.0

Environment

Python	3.14.7
Platform	Windows-11-10.0.26300-SP0
Packages	• pytest: 9.2.0 • pluggy: 1.7.0
Plugins	• cov: 7.1.0 • html: 4.2.0 • metadata: 3.1.1

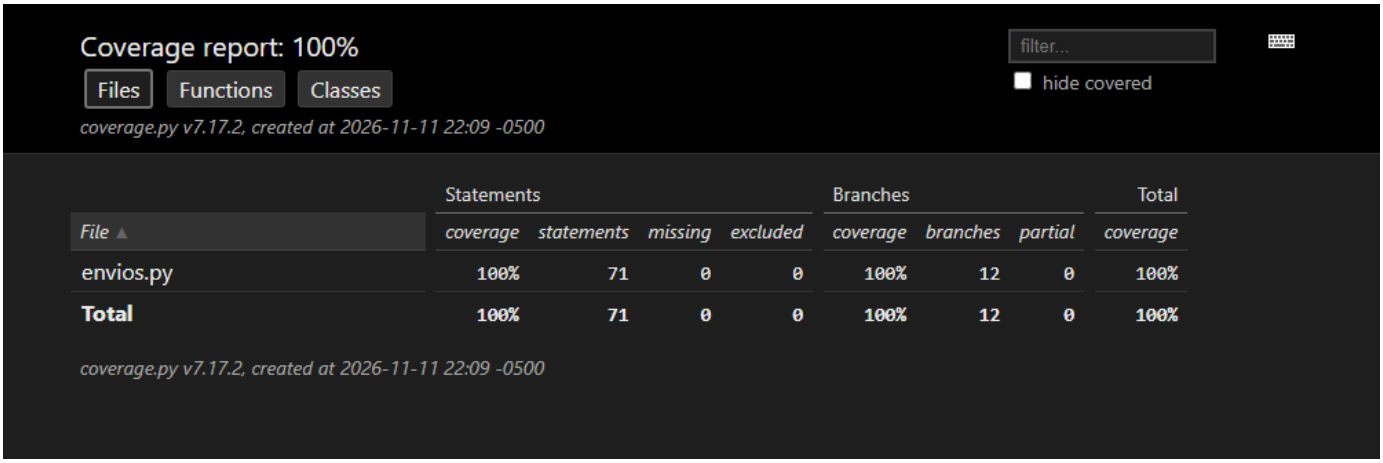
Summary

88 tests took 489 ms.

(Un)check the boxes to filter the results.

0 Failed, 88 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_caja_blanca.py::test_CB_01_peso_cero_lanza_error	2 ms	
Passed	tests/test_caja_blanca.py::test_CB_02_peso_sobre_el_tope_del_codigo_lanza_error	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_03_camino_base	1 ms	
Passed	tests/test_caja_blanca.py::test_CB_04_tramo_1	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_05_tramo_3	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_06_zona_rural	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_07_zona_nacional	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_09_fin_de_semana	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_10_cliente_frecuente	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_11_cliente_vip	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_12_cupon_envio10	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_14_costo_minimo	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_15_flag_ya_no_existe	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-03]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-04]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-05]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-06]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-07]	0 ms	



6.6 Antes y después

Métrica	v1.0.0 (heredado)	v1.0.1 (corregido)	v2.0.0 (refactorizado)
Complejidad ciclomática de la función principal	15	17	**1** (`calcular_costo`)
Complejidad ciclomática máxima de una función	15	17	**6** (`validar_envio`)
Complejidad promedio por bloque	15	17	**1,75**
Funciones y métodos	1	1	12
Función más larga (líneas)	30	36	**10**
Líneas del archivo (LOC)	42	49	135
Líneas de código sin comentarios ni blancos (SLOC)	37	44	79
Números mágicos en la lógica	9	12	0 (solo quedan el 100 de "por ciento" y el 0 de "peso positivo")
Cobertura de sentencias	100 % (29/29)	100 % (33/33)	100 % (71/71)
Cobertura de ramas	100 % (26/26)	100 % (28/28)	100 % (12/12)
Pruebas que pasan	37 de 57	57 de 57	88 de 88

Lectura:

- Corregir los defectos **subió** la complejidad de 15 a 17: cada validación nueva era un `if` más en la misma función. Por eso se refactorizó después.
- La refactorización **aumentó** las líneas (de 49 a 135) pero bajó la complejidad de cada pieza: ninguna función pasa de 6 y la más larga tiene 10 líneas. Más líneas, pero cada una más fácil de leer, probar y cambiar.
- Las ramas bajaron de 28 a 12 porque las decisiones sobre la zona y el cliente ya no son `if`: las resuelven el diccionario de estrategias y la tabla de descuentos.
- La cobertura se mantuvo en 100 % en las tres versiones.

Parte 7 - Gestión de configuración

Repositorio: https://github.com/alvarojaviermonterofonseca-ux/parcial-3-de-inegenieria-de-software

7.1 Línea base: etiqueta v1.0.0

El primer commit de trabajo contiene el código heredado tal como se recibió, con un único cambio: `CONFIG` lleno con mis valores. Ese commit lleva la etiqueta anotada **`v1.0.0`** y es la línea base contra la que se ejecutaron las pruebas de la Parte 5.

```
git tag -a v1.0.0 -m "v1.0.0 - código heredado sin modificar"
```

7.2 Ramas e integración

`main` siempre queda en un estado que se puede entregar. Cada trabajo se hizo en su propia rama y se integró con `git merge --no-ff`, para que en el historial quede visible qué commits pertenecen a cada rama.

Rama	Para qué	Se integró en
`test/suite`	Casos de caja negra y de caja blanca automatizados	`main`
`fix/DEF-04`	El sábado no cobraba recargo	`main`
`fix/DEF-06`	Truncaba en vez de redondear	`main`
`fix/DEF-01`	Aceptaba más de 45 kg	`main`
`fix/DEF-02`	Zona inexistente	`main`
`fix/DEF-03`	Día sin validar	`main`
`fix/DEF-05`	Cupón inexistente	`main`
`fix/DEF-07`	Mensaje de error	`main`
`refactor/estrategia`	Extracción de funciones, constantes y patrón Estrategia	`main`
`feature/hora-pico`	Solicitud de cambio CR-02	`main`

Reglas que seguí:

- Una rama por defecto, creada desde `main` actualizado. Las ramas `fix/` se hicieron en orden de **prioridad** del defecto, no en orden de número.
- Antes de integrar una rama se ejecutó toda la suite; solo se integró si no rompía ninguna prueba que ya pasaba.
- Mensajes de commit con prefijo según el tipo de cambio: `feat`, `fix`, `test`, `refactor`, `docs`, `chore`.

7.3 Versiones y justificación (versionado semántico)

En versionado semántico la versión es `MAJOR.MINOR.PATCH`: PATCH para correcciones compatibles, MINOR para funcionalidad nueva compatible y MAJOR cuando se rompe la interfaz pública.

Etiqueta	Qué contiene	Tipo	Justificación
`v1.0.0`	Código heredado	-	Línea base.
`v1.0.1`	Los 7 defectos corregidos	**PATCH**	Solo son correcciones para que el código cumpla la especificación. La firma `calc(p, z, c, d, cup=None, flag=False)` es exactamente la misma.
`v2.0.0`	Refactorización y patrón Estrategia	**MAJOR**	Se rompe la interfaz pública (ver abajo).
`v2.1.0`	Hora pico	**MINOR**	Funcionalidad nueva y compatible: `hora` es opcional.

¿Quitar `flag` hace que la versión sea major? Sí.

La interfaz pública de este módulo es la firma de `calc` y lo que se puede importar de `envios`. En la versión 2.0.0:

1. Se eliminó el parámetro `flag`. Un programa que llamaba `calc(5, "urbana", "normal", 1, None, True)` o `calc(..., flag=True)` funcionaba en 1.x y en 2.0.0 falla con `TypeError`. Eso es un cambio **incompatible hacia atrás**, y la regla 8 de SemVer dice que cualquier cambio incompatible en la API pública obliga a subir MAJOR, aunque el cambio sea pequeño o el parámetro casi no se usara.
2. Se cambiaron los nombres de los parámetros (`p` → `peso`, `z` → `zona`…): quien llamara con nombres (`calc(p=5, z="urbana", …)`) también se rompe.
3. Se eliminó el diccionario `CONFIG`: quien lo importara o lo modificara también se rompe.

Lo que **no** obliga a major: las llamadas posicionales normales (`calc(5, "urbana", "normal", 1)`) siguen funcionando, porque `calc` se dejó como alias de `calcular_costo`.

¿Y los defectos corregidos? Ahora se lanza error con entradas que antes devolvían un valor (zona inexistente, día 8, cupón inexistente, peso de 46 a 50 kg). No lo conté como cambio incompatible porque la interfaz documentada (la especificación R1 a R8) siempre dijo que eso era error: corregir el código para que cumpla su contrato es PATCH.

7.4 CHANGELOG y notas de versión

El archivo `CHANGELOG.md` sigue el formato *Keep a Changelog*: una sección por versión, de la más nueva a la más vieja, con los cambios agrupados en **Añadido**, **Cambiado**, **Eliminado** y **Corregido**, y los enlaces de comparación al final.

Notas de la versión 2.0.0

> **EnvíaYa - cálculo de costo de envíos 2.0.0** (11 de noviembre de 2026)
>
> Esta versión corrige el cobro y reorganiza el código por dentro.
>
> **Qué cambia para el negocio** (viene de 1.0.1)
>
> - Los sábados ya se cobra el recargo de fin de semana de $2.000.
> - El costo se redondea a los $100 más cercanos; antes siempre se redondeaba hacia abajo.
> - Ya no se aceptan envíos de más de 45 kg, zonas que no existen, días fuera de 1 a 7 ni cupones que no existen: el sistema avisa con un error.
>
> **Qué cambia para quien usa la función**
>
> - **Cambio incompatible:** desapareció el parámetro `flag`. Si necesita ver el costo, imprima el valor que devuelve la función.
> - **Cambio incompatible:** desapareció `CONFIG`; use las constantes del módulo.
> - La función se llama ahora `calcular_costo`; `calc` sigue funcionando.
>
> **Cómo actualizar**
>
> 1. Busque en su código `flag=` y llamadas a `calc` con seis argumentos, y quite ese argumento.
> 2. Si usaba nombres de parámetro (`p=`, `z=`, `c=`, `d=`, `cup=`), cámbielos por `peso=`, `zona=`, `cliente=`, `dia=`, `cupon=`.
>
> **Calidad:** 88 pruebas automatizadas, todas pasan; cobertura de sentencias y de ramas del 100 %.

Notas de la versión 2.1.0

> **EnvíaYa - cálculo de costo de envíos 2.1.0** (11 de noviembre de 2026)
>
> **Novedad: recargo de hora pico.** Los envíos que se hacen entre las 17 y las 19 (inclusive) tienen un recargo del 8 %, calculado sobre el valor con recargo de zona.
>
> - Para usarlo, envíe la hora por nombre: `calcular_costo(5, "rural", "normal", 1, hora=18)`.
> - Si no envía la hora, todo funciona igual que en la 2.0.0: **no hay que cambiar nada para actualizar.**
> - Una hora fuera de 0 a 23 produce el error "hora inválida".

> **Calidad:** 103 pruebas automatizadas, todas pasan; cobertura del 100 %.

7.5 Salida de `git log --oneline --graph --all`

```
* 0255145 docs: Partes 7 y 8, CHANGELOG 2.1.0, README e informe en PDF
*   8f2d3d1 merge: integra feature/hora-pico (CR-02)
|\
| * 021fee4 feat: recargo de hora pico del 8 % entre las 17 y las 19 (CR-02)
| * eb1d526 test: pruebas de hora pico escritas primero (CR-02, fallan)
|/
* 2091e45 docs: Parte 6, CHANGELOG y evidencias de la versión 2.0.0
*   b345a69 merge: integra refactor/estrategia
|\
| * 9eaa0b6 refactor: patrón Estrategia para el recargo por zona
| * f044c12 refactor: extrae funciones, nombra las constantes y elimina flag
* | fd2e4bc docs: CHANGELOG y evidencias de regresión de la versión 1.0.1
* | 1c48d51 merge: integra fix/DEF-07
|\|
| * 059f836 fix(DEF-07): el mensaje de error dice "peso inválido"
* | 98f7092 merge: integra fix/DEF-05
|\|
| * 23cbbf9 fix(DEF-05): un cupón que no existe lanza error
* | 26e24b7 merge: integra fix/DEF-03
|\|
| * bfc34e9 fix(DEF-03): valida que el día esté entre 1 y 7
* | e6413ae merge: integra fix/DEF-02
|\|
| * 91e9476 fix(DEF-02): una zona que no existe lanza error
|/
* 73744a4 docs: captura del reporte de pytest fallando contra v1.0.0 (Parte 5)
*   5f89973 merge: integra fix/DEF-01
|\
| * 3da56cb fix(DEF-01): rechaza envíos de más de 45 kg
|/
* a9de32d docs: capturas del reporte HTML de cobertura sobre v1.0.0 (Parte 5)
*   9f738b1 merge: integra fix/DEF-06
|\
| * 99de1d2 fix(DEF-06): redondea al múltiplo de $100 más cercano
|/
*   bc7f806 merge: integra fix/DEF-04
|\
| * 7182ef7 fix(DEF-04): el sábado cobra el recargo de fin de semana
|/
* 5dd9bec docs: reporte de 7 defectos, cobertura y métricas contra v1.0.0 (Parte 5)
*   0b41a3d merge: integra test/suite (casos de caja negra y caja blanca)
|\
| * 2573229 test: 15 caminos independientes de caja blanca y grafo de flujo (Parte 4)
| * b9760bb test: 42 casos de caja negra con pytest (Parte 3)
|/
* 3399948 docs: plan de pruebas con niveles, estrategia, riesgos y criterios (Parte 2)
* e7ef91e docs: especificación R1-R8, historias de usuario y trazabilidad (Parte 1)
* 05a7957 feat: código heredado de EnvíaYa con mi CONFIG (v1.0.0)
* 5293245 Initial commit
```

7.6 Distribución de los commits en el tiempo

Fecha y hora de cada commit (`git log --date=format:'%H:%M' --format='%ad %s'`):

```
18:59 Initial commit
19:44 feat: código heredado de EnvíaYa con mi CONFIG (v1.0.0)
19:47 docs: especificación R1-R8, historias de usuario y trazabilidad (Parte 1)
19:56 docs: plan de pruebas con niveles, estrategia, riesgos y criterios (Parte 2)
20:07 test: 42 casos de caja negra con pytest (Parte 3)
20:17 test: 15 caminos independientes de caja blanca y grafo de flujo (Parte 4)
20:26 merge: integra test/suite (casos de caja negra y caja blanca)
20:36 docs: reporte de 7 defectos, cobertura y métricas contra v1.0.0 (Parte 5)
20:46 fix(DEF-04): el sábado cobra el recargo de fin de semana
20:46 merge: integra fix/DEF-04
20:56 fix(DEF-06): redondea al múltiplo de $100 más cercano
20:56 merge: integra fix/DEF-06
21:01 docs: capturas del reporte HTML de cobertura sobre v1.0.0 (Parte 5)
21:06 fix(DEF-01): rechaza envíos de más de 45 kg
21:06 merge: integra fix/DEF-01
21:08 docs: captura del reporte de pytest fallando contra v1.0.0 (Parte 5)
21:17 fix(DEF-02): una zona que no existe lanza error
21:27 fix(DEF-03): valida que el día esté entre 1 y 7
```

```
21:36 fix(DEF-05): un cupón que no existe lanza error
21:44 fix(DEF-07): el mensaje de error dice "peso inválido"
21:57 refactor: extrae funciones, nombra las constantes y elimina flag
22:09 merge: integra fix/DEF-02
22:09 merge: integra fix/DEF-03
22:09 refactor: patrón Estrategia para el recargo por zona
22:09 merge: integra fix/DEF-05
22:09 merge: integra fix/DEF-07
22:09 docs: CHANGELOG y evidencias de regresión de la versión 1.0.1
22:09 merge: integra refactor/estrategia
22:09 docs: Parte 6, CHANGELOG y evidencias de la versión 2.0.0
22:09 test: pruebas de hora pico escritas primero (CR-02, fallan)
22:09 feat: recargo de hora pico del 8 % entre las 17 y las 19 (CR-02)
22:09 merge: integra feature/hora-pico (CR-02)
22:10 docs: Partes 7 y 8, CHANGELOG 2.1.0, README e informe en PDF
```

Parte 8 - Solicitud de cambio y análisis de impacto

8.1 La solicitud que me corresponde

(A + F) mod 3 = (3 + 5) mod 3 = 8 mod 3 = **2**

> **CR-02 - Hora pico.** Se agrega el parámetro `hora` (de 0 a 23). Entre las 17 y las 19, inclusive, hay un recargo del 8 %, que se calcula después del recargo por zona.

La escribí como una regla nueva de la especificación:

ID	Regla
R9	La función recibe `hora`, un entero de 0 a 23. Si la hora está entre 17 y 19, inclusive, se suma un recargo del **8 %** sobre el valor que resulta de aplicar el recargo por zona. Una hora fuera de 0 a 23 es error.

Decisiones que tomé donde la solicitud no dice nada:

1. `hora` **es opcional.** Si no se envía, no se cobra hora pico. Así los programas que ya llaman a la función siguen funcionando igual.
2. `hora` **solo se acepta por nombre** (`hora=18`). Antes, el sexto argumento posicional era `flag`; si `hora` ocupara ese lugar, una llamada vieja como `calc(5, "urbana", "normal", 1, None, True)` se interpretaría en silencio como "hora 1". Al exigir el nombre, esa llamada sigue dando `TypeError`.
3. **Orden.** El enunciado dice "después del recargo por zona". Con mis reglas el orden completo queda: tarifa base → zona → **hora pico** → fin de semana → descuentos → mínimo → redondeo. El recargo fijo de fin de semana no entra en la base del 8 %.
4. **Una hora no entera** (por ejemplo 18,5) se rechaza: la regla habla de horas de 0 a 23.

8.2 Análisis de impacto con la matriz de trazabilidad

Se recorrió la matriz de la Parte 1 preguntando, requisito por requisito, si el cambio lo toca.

Elemento	¿Se afecta?	Detalle
R1 peso	No	-
R2 zona y día válidos	Sí (se amplía)	La validación ahora incluye la hora (0 a 23).
R3 tarifa base	No	-
R4 recargo por zona	Indirecto	No cambia, pero su resultado es la base del nuevo recargo. Los casos de zona sirven de regresión.
R5 fin de semana	Indirecto	Hay que fijar el orden: la hora pico va antes de sumar los $2.000.
R6 descuentos	Indirecto	Los descuentos se aplican sobre un valor que ahora puede traer el 8 %.
R7 cupón	No	-
R8 mínimo y redondeo	Indirecto	El 8 % produce valores con centavos (5.248,80): el redondeo debe seguir siendo correcto.
R9 hora pico	**Nuevo**	Regla nueva.

Historias	¿Se afecta?	Detalle
HU-01 validar datos	Sí	Nuevo criterio: hora fuera de 0 a 23 → error.
HU-02 peso y zona	No	Sus casos se vuelven a ejecutar como regresión.
HU-03 fin de semana y descuentos	No (regresión)	Se agrega un caso que combina hora pico, domingo y VIP para fijar el orden.
HU-04 hora pico	**Nueva**	Ver abajo.

HU-04 - Cobrar recargo en hora pico (R9)

> **Como** gerente de operaciones de EnvíaYa, **quiero** cobrar un recargo del 8 % a los envíos que se hacen entre las 17 y las 19, **para** compensar el mayor costo de despachar en las horas de más tráfico.

- **CA-04.1 Dado** un envío de 5 kg urbano un lunes, **cuando** se hace a las 17, 18 o 19, **entonces** cuesta $9.700 (9.000 × 1,08 = 9.720).
- **CA-04.2 Dado** el mismo envío, **cuando** se hace a las 16 o a las 20, **entonces** cuesta $9.000.
- **CA-04.3 Dado** un envío de 5 kg a zona rural un lunes a las 18, **cuando** calculo el costo, **entonces** el 8 % se calcula sobre el valor con recargo de zona: 9.000 × 1,20 × 1,08 = 11.664 → $11.700.
- **CA-04.4 Dado** un envío con hora −1 o 24, **cuando** calculo el costo, **entonces** el sistema lanza un error.
- **CA-04.5 Dado** un envío sin hora, **cuando** calculo el costo, **entonces** el resultado es el mismo de antes del cambio.

Casos de prueba	Impacto
CN-01 a CN-42, CB-01 a CB-15, pruebas de unidad	**No cambian.** Ninguno envía hora, así que todos deben seguir pasando: son la regresión.
HP-01 a HP-15	**Nuevos** (`tests/test_hora_pico.py`).

Código	Impacto
Constantes	Nuevas: `HORAS_VALIDAS`, `HORA_PICO_INICIO`, `HORA_PICO_FIN`, `RECARGO_HORA_PICO_PCT`.
`validar_envio`	Se modifica: valida la hora.
`aplicar_hora_pico`	Función nueva.
`calcular_costo`	Se modifica: nuevo parámetro `hora` y un paso más en el cálculo.
`tarifa_base`, estrategias de zona, `recargo_fin_de_semana`, `aplicar_descuentos`, `redondear`	**No se tocan.**

Fila nueva de la matriz de trazabilidad:

Requisito	Historia	Criterios	Casos de prueba
R9 hora pico	HU-04 (y HU-01 por la validación)	CA-04.1 a CA-04.5	HP-01 a HP-15

Tamaño del impacto: bajo. Gracias a la refactorización el cambio se concentra en una función nueva de 5 líneas y en dos funciones que se modifican; en la versión 1.0.0 habría sido otro `if` dentro de una función de complejidad 15.

Riesgo principal: aplicar el 8 % en el lugar equivocado del cálculo. Se controla con los casos HP-10 y HP-11, que dan un resultado distinto si el orden está mal.

8.3 Tipo de mantenimiento

Perfectivo. Es una funcionalidad nueva pedida por el negocio: el sistema no tenía una falla (no es correctivo) ni cambió el entorno técnico o legal en el que corre (no es adaptativo). Sería adaptativo si el recargo lo impusiera, por ejemplo, una norma de tránsito externa a la empresa.

8.4 Implementación: las pruebas primero

Rama `feature/hora-pico`.

Paso 1 - Escribir las pruebas y verlas fallar. Se diseñaron los casos con las mismas técnicas de la Parte 3. Base: 5 kg urbano lunes = $9.000.

ID	Entradas	Hora	Cálculo a mano	Esperado	Técnica
HP-01	5 kg, urbana, normal, lunes	16	9.000	$9.000	VL (17)
HP-02	5 kg, urbana, normal, lunes	17	9.000 × 1,08 = 9.720	$9.700	VL (17)

ID	Entradas	Hora	Cálculo a mano	Esperado	Técnica
HP-03	5 kg, urbana, normal, lunes	18	9.720	$9.700	PE (pico)
HP-04	5 kg, urbana, normal, lunes	19	9.720	$9.700	VL (19)
HP-05	5 kg, urbana, normal, lunes	20	9.000	$9.000	VL (19)
HP-06	5 kg, urbana, normal, lunes	0	9.000	$9.000	VL (0)
HP-07	5 kg, urbana, normal, lunes	23	9.000	$9.000	VL (23)
HP-08	5 kg, urbana, normal, lunes	−1	-	Error	VL (0)
HP-09	5 kg, urbana, normal, lunes	24	-	Error	VL (23)
HP-10	5 kg, rural, normal, lunes	18	9.000 × 1,20 = 10.800; × 1,08 = 11.664	$11.700	Orden: después de zona
HP-11	5 kg, urbana, vip, domingo	17	9.720 + 2.000 = 11.720; × 0,90 = 10.548	$10.500	Orden: antes de fin de semana
HP-12	12,5 kg, nacional, vip, sábado, ENVIO10	19	11.000 × 1,50 = 16.500; × 1,08 = 17.820; + 2.000 = 19.820; × 0,90 = 17.838; × 0,90 = 16.054,2	$16.100	Combinación
HP-13	2 kg, urbana, vip, lunes, ENVIO10	18	6.000 × 1,08 = 6.480; × 0,90 × 0,90 = 5.248,8	$5.200	VL (costo mínimo)
HP-14	5 kg, urbana, normal, lunes	18,5	-	Error	PE (hora no entera)
HP-15	Sin hora	-	Igual que antes	$9.000 y $10.400	Compatibilidad

Por qué HP-10 y HP-11 fijan el orden:

- **HP-10.** Si el 8 % se calculara sobre la tarifa base y no sobre el valor con zona: 10.800 + 9.000 × 0,08 = 11.520 → $11.500 (no $11.700).
- **HP-11.** Si el 8 % se aplicara después de sumar el fin de semana: (9.000 + 2.000) × 1,08 × 0,90 = 10.692 → $10.700 (no $10.500).

Las pruebas se confirmaron **antes** de tocar `envios.py`. Resultado: fallan las que usan `hora`, porque el parámetro todavía no existe:

```
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:50: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
========================== short test summary info ==========================
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-01] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-02] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-03] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-04] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-05] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-06] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-07] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-10] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-11] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-12] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-13] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_hora_invalida_lanza_error[HP-08] - TypeE...
FAILED tests/test_hora_pico.py::test_hora_invalida_lanza_error[HP-09] - TypeE...
FAILED tests/test_hora_pico.py::test_hora_invalida_lanza_error[HP-14] - TypeE...
14 failed, 1 passed in 0.05s
```

Paso 2 - Implementar.

```
HORAS_VALIDAS = range(0, 24)
HORA_PICO_INICIO = 17           # incluida
HORA_PICO_FIN = 19              # incluida
RECARGO_HORA_PICO_PCT = 8

def aplicar_hora_pico(valor, hora):
    """R9: +8 % entre las 17 y las 19, sobre el valor que ya trae la zona."""
    if hora is not None and HORA_PICO_INICIO <= hora <= HORA_PICO_FIN:
        return sumar_porcentaje(valor, RECARGO_HORA_PICO_PCT)
    return valor

def calcular_costo(peso, zona, cliente, dia, cupon=None, *, hora=None):
    validar_envio(peso, zona, dia, cupon, hora)
    valor = tarifa_base(peso)
    valor = ZONAS[zona].aplicar_recargo(valor)
    valor = aplicar_hora_pico(valor, hora)        # después de la zona
    valor = valor + recargo_fin_de_semana(dia)    # antes de los descuentos
    valor = aplicar_descuentos(valor, cliente, cupon)
    return redondear(valor)
```

Paso 3 - Regresión completa. Pasan las 15 pruebas nuevas y las 88 anteriores, sin modificar ninguna:

```
============================== tests coverage ==============================
_________________ coverage: platform win32, python 3.14.7-final-0 _________________

Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
envios.py      82      0     16      0   100%
-----------------------------------------------------------
TOTAL          82      0     16      0   100%
103 passed in 0.57s
```

Capturas sobre la versión 2.1.0: reporte de pytest con las 103 pruebas en verde y reporte de cobertura.

pruebas-v2.1.0.html

Report generated on 11-Nov-2026 at 22:14:46 by pytest-html v4.2.0

Environment

Python	3.14.7
Platform	Windows-11-10.0.26300-SP0
Packages	• pytest: 9.2.0 • pluggy: 1.7.0
Plugins	• cov: 7.1.0 • html: 4.2.0 • metadata: 3.1.1

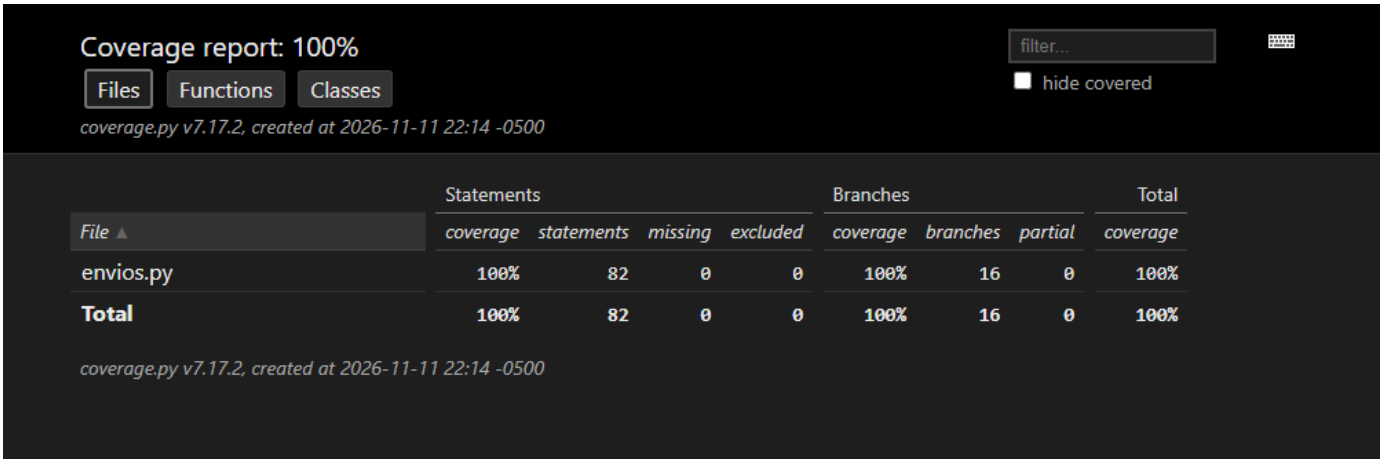
Summary

103 tests took 646 ms.

(Un)check the boxes to filter the results.

0 Failed, 103 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_caja_blanca.py::test_CB_01_peso_cero_lanza_error	8 ms	
Passed	tests/test_caja_blanca.py::test_CB_02_peso_sobre_el_tope_del_codigo_lanza_error	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_03_camino_base	1 ms	
Passed	tests/test_caja_blanca.py::test_CB_04_tramo_1	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_05_tramo_3	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_06_zona_rural	1 ms	
Passed	tests/test_caja_blanca.py::test_CB_07_zona_nacional	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_09_fin_de_semana	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_10_cliente_frecuente	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_11_cliente_vip	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_12_cupon_envio10	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_14_costo_minimo	0 ms	
Passed	tests/test_caja_blanca.py::test_CB_15_flag_ya_no_existe	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-03]	1 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-04]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-05]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-06]	0 ms	
Passed	tests/test_caja_negra.py::test_costo_esperado[CN-07]	0 ms	



8.5 Nueva versión

Versión anterior	2.0.0
Versión nueva	**2.1.0**
Por qué MINOR	Se agrega funcionalidad **compatible**: `hora` es opcional y con nombre; toda llamada que funcionaba en 2.0.0 da el mismo resultado en 2.1.0 (lo demuestra la regresión). No es PATCH porque no es una corrección, y no es MAJOR porque no rompe a nadie.
Etiqueta	`v2.1.0`
CHANGELOG	Sección `[2.1.0]` con el cambio en "Añadido".

Métricas después del cambio:

Métrica	**v2.0.0**	**v2.1.0**
Pruebas	88	103
Pruebas que pasan	88	103
Cobertura de sentencias	100 % (71/71)	100 % (82/82)
Cobertura de ramas	100 % (12/12)	100 % (16/16)
Complejidad máxima	6	8 (`validar_envio`)
Complejidad promedio	1,75	1,94

Anexo - Evidencias de ejecución

Salidas reales de la consola, guardadas en `docs/evidencias/` del repositorio.

A.1 Pruebas contra el código heredado v1.0.0 (fallan 20)

```
$ git describe --tags
v1.0.0
$ python -m pytest -v --tb=line --cov=envios --cov-branch --cov-report=term-missing
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\AppData\Local\Programs\Python\Python314\python.exe
rootdir: C:\Users\Acer\Downloads\parcial de ingeneria\repo
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0
collecting ... collected 57 items

tests/test_caja_blanca.py::test_CB_01_peso_cero_lanza_error PASSED       [  1%]
tests/test_caja_blanca.py::test_CB_02_peso_sobre_el_tope_del_codigo_lanza_error PASSED [  3%]
tests/test_caja_blanca.py::test_CB_03_camino_base PASSED                 [  5%]
tests/test_caja_blanca.py::test_CB_04_tramo_1 PASSED                     [  7%]
tests/test_caja_blanca.py::test_CB_05_tramo_3 PASSED                     [  8%]
tests/test_caja_blanca.py::test_CB_06_zona_rural PASSED                  [ 10%]
tests/test_caja_blanca.py::test_CB_07_zona_nacional PASSED               [ 12%]
tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe FAILED          [ 14%]
tests/test_caja_blanca.py::test_CB_09_fin_de_semana PASSED               [ 15%]
tests/test_caja_blanca.py::test_CB_10_cliente_frecuente FAILED           [ 17%]
tests/test_caja_blanca.py::test_CB_11_cliente_vip PASSED                 [ 19%]
tests/test_caja_blanca.py::test_CB_12_cupon_envio10 PASSED               [ 21%]
tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe FAILED         [ 22%]
tests/test_caja_blanca.py::test_CB_14_costo_minimo PASSED                [ 24%]
tests/test_caja_blanca.py::test_CB_15_flag_imprime_el_costo PASSED       [ 26%]
tests/test_caja_negra.py::test_costo_esperado[CN-03] PASSED              [ 28%]
tests/test_caja_negra.py::test_costo_esperado[CN-04] PASSED              [ 29%]
tests/test_caja_negra.py::test_costo_esperado[CN-05] PASSED              [ 31%]
tests/test_caja_negra.py::test_costo_esperado[CN-06] PASSED              [ 33%]
tests/test_caja_negra.py::test_costo_esperado[CN-07] PASSED              [ 35%]
tests/test_caja_negra.py::test_costo_esperado[CN-08] PASSED              [ 36%]
tests/test_caja_negra.py::test_costo_esperado[CN-09] PASSED              [ 38%]
tests/test_caja_negra.py::test_costo_esperado[CN-10] PASSED              [ 40%]
tests/test_caja_negra.py::test_costo_esperado[CN-11] FAILED              [ 42%]
tests/test_caja_negra.py::test_costo_esperado[CN-12] PASSED              [ 43%]
tests/test_caja_negra.py::test_costo_esperado[CN-15] PASSED              [ 45%]
tests/test_caja_negra.py::test_costo_esperado[CN-16] PASSED              [ 47%]
tests/test_caja_negra.py::test_costo_esperado[CN-19] PASSED              [ 49%]
tests/test_caja_negra.py::test_costo_esperado[CN-20] PASSED              [ 50%]
tests/test_caja_negra.py::test_costo_esperado[CN-21] FAILED              [ 52%]
tests/test_caja_negra.py::test_costo_esperado[CN-22] PASSED              [ 54%]
tests/test_caja_negra.py::test_costo_esperado[CN-24] PASSED              [ 56%]
tests/test_caja_negra.py::test_costo_esperado[CN-25] FAILED              [ 57%]
tests/test_caja_negra.py::test_costo_esperado[CN-26] FAILED              [ 59%]
tests/test_caja_negra.py::test_costo_esperado[CN-27] PASSED              [ 61%]
tests/test_caja_negra.py::test_costo_esperado[CN-28] FAILED              [ 63%]
tests/test_caja_negra.py::test_costo_esperado[CN-29] PASSED              [ 64%]
tests/test_caja_negra.py::test_costo_esperado[CN-30] FAILED              [ 66%]
tests/test_caja_negra.py::test_costo_esperado[CN-31] PASSED              [ 68%]
tests/test_caja_negra.py::test_costo_esperado[CN-32] PASSED              [ 70%]
tests/test_caja_negra.py::test_costo_esperado[CN-33] PASSED              [ 71%]
tests/test_caja_negra.py::test_costo_esperado[CN-36] PASSED              [ 73%]
tests/test_caja_negra.py::test_costo_esperado[CN-37] PASSED              [ 75%]
tests/test_caja_negra.py::test_costo_esperado[CN-38] FAILED              [ 77%]
tests/test_caja_negra.py::test_costo_esperado[CN-39] FAILED              [ 78%]
tests/test_caja_negra.py::test_costo_esperado[CN-40] PASSED              [ 80%]
tests/test_caja_negra.py::test_costo_esperado[CN-41] FAILED              [ 82%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-01] PASSED [ 84%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-02] PASSED [ 85%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-13] FAILED [ 87%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-14] FAILED [ 89%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-17] FAILED [ 91%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-18] FAILED [ 92%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-23] FAILED [ 94%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-34] FAILED [ 96%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-35] FAILED [ 98%]
tests/test_caja_negra.py::test_CN_42_mensaje_de_peso_invalido FAILED     [100%]

================================== FAILURES ===================================
E   Failed: DID NOT RAISE ValueError
```

```
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_blanca.py:55: Failed: DID NOT RAISE ValueError
E   AssertionError: assert 8500 == 8600
     +  where 8500 = calc(5, 'urbana', 'frecuente', 1)
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_blanca.py:66: AssertionError: assert 8500 == 8600
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_blanca.py:82: Failed: DID NOT RAISE ValueError
E   AssertionError: assert 36900 == 37000
     +  where 36900 = calc(44.99, 'urbana', 'normal', 1, None)
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 36900 == 37000
E   AssertionError: assert 9000 == 11000
     +  where 9000 = calc(5, 'urbana', 'normal', 6, None)
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 9000 == 11000
E   AssertionError: assert 8500 == 8600
     +  where 8500 = calc(5, 'urbana', 'frecuente', 1, None)
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 8500 == 8600
E   AssertionError: assert 7600 == 7700
     +  where 7600 = calc(5, 'urbana', 'frecuente', 1, 'ENVIO10')
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 7600 == 7700
E   AssertionError: assert 7200 == 7300
     +  where 7200 = calc(5, 'urbana', 'vip', 1, 'ENVIO10')
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 7200 == 7300
E   AssertionError: assert 10400 == 10500
     +  where 10400 = calc(5, 'urbana', 'frecuente', 7, None)
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 10400 == 10500
E   AssertionError: assert 8700 == 10400
     +  where 8700 = calc(4, 'rural', 'vip', 6, 'ENVIO10')
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 8700 == 10400
E   AssertionError: assert 13300 == 15000
     +  where 13300 = calc(12.5, 'nacional', 'vip', 6, 'ENVIO10')
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 13300 == 15000
E   AssertionError: assert 6800 == 8700
     +  where 6800 = calc(3, 'rural', 'frecuente', 6, None)
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:80: AssertionError: assert 6800 == 8700
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:89: Failed: DID NOT RAISE ValueError
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:89: Failed: DID NOT RAISE ValueError
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:89: Failed: DID NOT RAISE ValueError
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:89: Failed: DID NOT RAISE ValueError
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:89: Failed: DID NOT RAISE ValueError
E   Failed: DID NOT RAISE ValueError
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:89: Failed: DID NOT RAISE ValueError
E   AssertionError: assert 'peso invalido' == 'peso inválido'

      - peso inválido
      ?         ^
      + peso invalido
      ?         ^
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_caja_negra.py:97: AssertionError: assert 'peso invalido' == 'peso i
nválido'
============================== tests coverage ==============================
_________________ coverage: platform win32, python 3.14.7-final-0 _________________

Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
envios.py      29      0     26      0   100%
-----------------------------------------------------
TOTAL          29      0     26      0   100%
========================== short test summary info ==========================
FAILED tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe - Failed: DID...
FAILED tests/test_caja_blanca.py::test_CB_10_cliente_frecuente - AssertionErr...
FAILED tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe - Failed: DI...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-11] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-21] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-25] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-26] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-28] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-30] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-38] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-39] - AssertionError:...
FAILED tests/test_caja_negra.py::test_costo_esperado[CN-41] - AssertionError:...
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-13] - F...
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-14] - F...
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-17] - F...
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-18] - F...
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-23] - F...
```

```
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-34] - F...
FAILED tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-35] - F...
FAILED tests/test_caja_negra.py::test_CN_42_mensaje_de_peso_invalido - Assert...
====================== 20 failed, 37 passed in 0.25s ======================
```

A.2 Pruebas de regresión sobre v2.0.0 (todas pasan)

```
$ python -m pytest -v --tb=line --cov=envios --cov-branch --cov-report=term-missing
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Acer\AppData\Local\Programs\Python\Python314\python.exe
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26300-SP0', 'Packages': {'pytest': '9.2.0', 'pluggy': '1.7.0'}, 'Plu
gins': {'cov': '7.1.0', 'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\Acer\Downloads\parcial de ingeneria\repo
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0, html-4.2.0, metadata-3.1.1
collecting ... collected 88 items

tests/test_caja_blanca.py::test_CB_01_peso_cero_lanza_error PASSED       [  1%]
tests/test_caja_blanca.py::test_CB_02_peso_sobre_el_tope_del_codigo_lanza_error PASSED [  2%]
tests/test_caja_blanca.py::test_CB_03_camino_base PASSED                 [  3%]
tests/test_caja_blanca.py::test_CB_04_tramo_1 PASSED                     [  4%]
tests/test_caja_blanca.py::test_CB_05_tramo_3 PASSED                     [  5%]
tests/test_caja_blanca.py::test_CB_06_zona_rural PASSED                  [  6%]
tests/test_caja_blanca.py::test_CB_07_zona_nacional PASSED               [  7%]
tests/test_caja_blanca.py::test_CB_08_zona_que_no_existe PASSED          [  9%]
tests/test_caja_blanca.py::test_CB_09_fin_de_semana PASSED               [ 10%]
tests/test_caja_blanca.py::test_CB_10_cliente_frecuente PASSED           [ 11%]
tests/test_caja_blanca.py::test_CB_11_cliente_vip PASSED                 [ 12%]
tests/test_caja_blanca.py::test_CB_12_cupon_envio10 PASSED               [ 13%]
tests/test_caja_blanca.py::test_CB_13_cupon_que_no_existe PASSED         [ 14%]
tests/test_caja_blanca.py::test_CB_14_costo_minimo PASSED                [ 15%]
tests/test_caja_blanca.py::test_CB_15_flag_ya_no_existe PASSED           [ 17%]
tests/test_caja_negra.py::test_costo_esperado[CN-03] PASSED              [ 18%]
tests/test_caja_negra.py::test_costo_esperado[CN-04] PASSED              [ 19%]
tests/test_caja_negra.py::test_costo_esperado[CN-05] PASSED              [ 20%]
tests/test_caja_negra.py::test_costo_esperado[CN-06] PASSED              [ 21%]
tests/test_caja_negra.py::test_costo_esperado[CN-07] PASSED              [ 22%]
tests/test_caja_negra.py::test_costo_esperado[CN-08] PASSED              [ 23%]
tests/test_caja_negra.py::test_costo_esperado[CN-09] PASSED              [ 25%]
tests/test_caja_negra.py::test_costo_esperado[CN-10] PASSED              [ 26%]
tests/test_caja_negra.py::test_costo_esperado[CN-11] PASSED              [ 27%]
tests/test_caja_negra.py::test_costo_esperado[CN-12] PASSED              [ 28%]
tests/test_caja_negra.py::test_costo_esperado[CN-15] PASSED              [ 29%]
tests/test_caja_negra.py::test_costo_esperado[CN-16] PASSED              [ 30%]
tests/test_caja_negra.py::test_costo_esperado[CN-19] PASSED              [ 31%]
tests/test_caja_negra.py::test_costo_esperado[CN-20] PASSED              [ 32%]
tests/test_caja_negra.py::test_costo_esperado[CN-21] PASSED              [ 34%]
tests/test_caja_negra.py::test_costo_esperado[CN-22] PASSED              [ 35%]
tests/test_caja_negra.py::test_costo_esperado[CN-24] PASSED              [ 36%]
tests/test_caja_negra.py::test_costo_esperado[CN-25] PASSED              [ 37%]
tests/test_caja_negra.py::test_costo_esperado[CN-26] PASSED              [ 38%]
tests/test_caja_negra.py::test_costo_esperado[CN-27] PASSED              [ 39%]
tests/test_caja_negra.py::test_costo_esperado[CN-28] PASSED              [ 40%]
tests/test_caja_negra.py::test_costo_esperado[CN-29] PASSED              [ 42%]
tests/test_caja_negra.py::test_costo_esperado[CN-30] PASSED              [ 43%]
tests/test_caja_negra.py::test_costo_esperado[CN-31] PASSED              [ 44%]
tests/test_caja_negra.py::test_costo_esperado[CN-32] PASSED              [ 45%]
tests/test_caja_negra.py::test_costo_esperado[CN-33] PASSED              [ 46%]
tests/test_caja_negra.py::test_costo_esperado[CN-36] PASSED              [ 47%]
tests/test_caja_negra.py::test_costo_esperado[CN-37] PASSED              [ 48%]
tests/test_caja_negra.py::test_costo_esperado[CN-38] PASSED              [ 50%]
tests/test_caja_negra.py::test_costo_esperado[CN-39] PASSED              [ 51%]
tests/test_caja_negra.py::test_costo_esperado[CN-40] PASSED              [ 52%]
tests/test_caja_negra.py::test_costo_esperado[CN-41] PASSED              [ 53%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-01] PASSED [ 54%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-02] PASSED [ 55%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-13] PASSED [ 56%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-14] PASSED [ 57%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-17] PASSED [ 59%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-18] PASSED [ 60%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-23] PASSED [ 61%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-34] PASSED [ 62%]
tests/test_caja_negra.py::test_entrada_invalida_lanza_error[CN-35] PASSED [ 63%]
tests/test_caja_negra.py::test_CN_42_mensaje_de_peso_invalido PASSED     [ 64%]
tests/test_unidades.py::test_cada_zona_aplica_su_recargo[urbana] PASSED  [ 65%]
tests/test_unidades.py::test_cada_zona_aplica_su_recargo[rural] PASSED   [ 67%]
tests/test_unidades.py::test_cada_zona_aplica_su_recargo[nacional] PASSED [ 68%]
tests/test_unidades.py::test_todas_las_zonas_registradas_son_estrategias PASSED [ 69%]
```

```
tests/test_unidades.py::test_la_estrategia_abstracta_no_se_puede_instanciar PASSED [ 70%]
tests/test_unidades.py::test_tarifa_base[0.5-6000] PASSED                [ 71%]
tests/test_unidades.py::test_tarifa_base[3-6000] PASSED                  [ 72%]
tests/test_unidades.py::test_tarifa_base[3.01-9000] PASSED               [ 73%]
tests/test_unidades.py::test_tarifa_base[10-9000] PASSED                 [ 75%]
tests/test_unidades.py::test_tarifa_base[10.5-9400] PASSED               [ 76%]
tests/test_unidades.py::test_tarifa_base[45-37000] PASSED                [ 77%]
tests/test_unidades.py::test_recargo_fin_de_semana[1-0] PASSED           [ 78%]
tests/test_unidades.py::test_recargo_fin_de_semana[5-0] PASSED           [ 79%]
tests/test_unidades.py::test_recargo_fin_de_semana[6-2000] PASSED        [ 80%]
tests/test_unidades.py::test_recargo_fin_de_semana[7-2000] PASSED        [ 81%]
tests/test_unidades.py::test_descuentos_se_aplican_en_cadena PASSED      [ 82%]
tests/test_unidades.py::test_redondear[8549.99-8500] PASSED              [ 84%]
tests/test_unidades.py::test_redondear[8550-8600] PASSED                 [ 85%]
tests/test_unidades.py::test_redondear[8550.01-8600] PASSED              [ 86%]
tests/test_unidades.py::test_redondear[8649.99-8600] PASSED              [ 87%]
tests/test_unidades.py::test_redondear[4860-5000] PASSED                 [ 88%]
tests/test_unidades.py::test_redondear[5000-5000] PASSED                 [ 89%]
tests/test_unidades.py::test_redondear[5049.99-5000] PASSED              [ 90%]
tests/test_unidades.py::test_redondear[5050-5100] PASSED                 [ 92%]
tests/test_unidades.py::test_validar_envio_dice_que_dato_esta_mal[0-urbana-1-None-peso inv\xe1lido] PASSED [ 93%]
tests/test_unidades.py::test_validar_envio_dice_que_dato_esta_mal[45.01-urbana-1-None-peso inv\xe1lido] PASSED [ 94%]
tests/test_unidades.py::test_validar_envio_dice_que_dato_esta_mal[5-lunar-1-None-zona inv\xe1lida] PASSED [ 95%]
tests/test_unidades.py::test_validar_envio_dice_que_dato_esta_mal[5-urbana-0-None-d\xeda inv\xe1lido] PASSED [ 96%]
tests/test_unidades.py::test_validar_envio_dice_que_dato_esta_mal[5-urbana-6.5-None-d\xeda inv\xe1lido] PASSED [ 97%]
tests/test_unidades.py::test_validar_envio_dice_que_dato_esta_mal[5-urbana-1-NOEXISTE-cup\xf3n inv\xe1lido] PASSED [ 98%]
tests/test_unidades.py::test_validar_envio_acepta_un_envio_correcto PASSED [100%]

============================== tests coverage ==============================
_________________ coverage: platform win32, python 3.14.7-final-0 _________________

Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
envios.py      71      0     12      0   100%
-----------------------------------------------------
TOTAL          71      0     12      0   100%
============================ 88 passed in 0.18s ============================
```

A.3 Pruebas de la hora pico escritas primero (fallan)

```
$ python -m pytest tests/test_hora_pico.py --tb=line -q   (antes de implementar)
FFFFFFFFFFFFF.                                                           [100%]
================================== FAILURES ===================================
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:44: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:50: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
```

```
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:50: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
E   TypeError: calcular_costo() got an unexpected keyword argument 'hora'
C:\Users\Acer\Downloads\parcial de ingeneria\repo\tests\test_hora_pico.py:50: TypeError: calcular_costo() got an unexpected keyw
ord argument 'hora'
========================== short test summary info ==========================
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-01] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-02] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-03] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-04] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-05] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-06] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-07] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-10] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-11] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-12] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_costo_con_hora[HP-13] - TypeError: calcu...
FAILED tests/test_hora_pico.py::test_hora_invalida_lanza_error[HP-08] - TypeE...
FAILED tests/test_hora_pico.py::test_hora_invalida_lanza_error[HP-09] - TypeE...
FAILED tests/test_hora_pico.py::test_hora_invalida_lanza_error[HP-14] - TypeE...
14 failed, 1 passed in 0.05s
```

A.4 Pruebas sobre v2.1.0 (todas pasan)

```
$ python -m pytest -q --cov=envios --cov-branch --cov-report=term-missing
.......................................................................... [ 69%]
...............................                                          [100%]
============================== tests coverage ==============================
_________________ coverage: platform win32, python 3.14.7-final-0 _________________

Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
envios.py      82      0     16      0   100%
-----------------------------------------------------
TOTAL          82      0     16      0   100%
103 passed in 0.57s
```

A.5 Historial de Git

```
$ git log --oneline --graph --all
* 0255145 docs: Partes 7 y 8, CHANGELOG 2.1.0, README e informe en PDF
*   8f2d3d1 merge: integra feature/hora-pico (CR-02)
|\
| * 021fee4 feat: recargo de hora pico del 8 % entre las 17 y las 19 (CR-02)
| * eb1d526 test: pruebas de hora pico escritas primero (CR-02, fallan)
|/
* 2091e45 docs: Parte 6, CHANGELOG y evidencias de la versión 2.0.0
*   b345a69 merge: integra refactor/estrategia
|\
| * 9eaa0b6 refactor: patrón Estrategia para el recargo por zona
| * f044c12 refactor: extrae funciones, nombra las constantes y elimina flag
* | fd2e4bc docs: CHANGELOG y evidencias de regresión de la versión 1.0.1
* | 1c48d51 merge: integra fix/DEF-07
|\|
| * 059f836 fix(DEF-07): el mensaje de error dice "peso inválido"
* | 98f7092 merge: integra fix/DEF-05
|\|
| * 23cbbf9 fix(DEF-05): un cupón que no existe lanza error
* | 26e24b7 merge: integra fix/DEF-03
|\|
| * bfc34e9 fix(DEF-03): valida que el día esté entre 1 y 7
* | e6413ae merge: integra fix/DEF-02
|\|
| * 91e9476 fix(DEF-02): una zona que no existe lanza error
|/
* 73744a4 docs: captura del reporte de pytest fallando contra v1.0.0 (Parte 5)
*   5f89973 merge: integra fix/DEF-01
|\
| * 3da56cb fix(DEF-01): rechaza envíos de más de 45 kg
|/
* a9de32d docs: capturas del reporte HTML de cobertura sobre v1.0.0 (Parte 5)
*   9f738b1 merge: integra fix/DEF-06
|\
| * 99de1d2 fix(DEF-06): redondea al múltiplo de $100 más cercano
|/
```

```
*   bc7f806 merge: integra fix/DEF-04
|\
| * 7182ef7 fix(DEF-04): el sábado cobra el recargo de fin de semana
|/
* 5dd9bec docs: reporte de 7 defectos, cobertura y métricas contra v1.0.0 (Parte 5)
*   0b41a3d merge: integra test/suite (casos de caja negra y caja blanca)
|\
| * 2573229 test: 15 caminos independientes de caja blanca y grafo de flujo (Parte 4)
| * b9760bb test: 42 casos de caja negra con pytest (Parte 3)
|/
* 3399948 docs: plan de pruebas con niveles, estrategia, riesgos y criterios (Parte 2)
* e7ef91e docs: especificación R1-R8, historias de usuario y trazabilidad (Parte 1)
* 05a7957 feat: código heredado de EnvíaYa con mi CONFIG (v1.0.0)
* 5293245 Initial commit
```